Unicorn - Detaljerad Utvecklingsprompt för Replit 3.0

Version: 1.0
Datum: 2026-01-09
Syfte: Komplett implementeringsguide för att bygga Unicorn-systemet

---

Innehållsförteckning

1. Kontext och Bakgrund
2. Fas 1 - Detaljerad Implementation
3. Tekniska Specifikationer
4. Implementationsordning
5. Konkreta Exempel och Use Cases
6. Viktiga Principer och Regler
7. Checklista för Replit 3.0

---

1. Kontext och Bakgrund

1.1 Vad är Unicorn?

Unicorn är ett produktionssystem för rörlig entreprenad och service, specifikt byggt för företag som Kinab som arbetar med:

- **Tvättservice** av återvinnings- och pantrum hos butikskedjor
- **Underhåll** av sopkärl, matavfallsbehållare och relaterad utrustning
- **Schemalagd service** på fysiska objekt geografiskt spridda
- **Optimerad planering** av rutter och arbetstid

Problemet som Unicorn Löser

Idag hos Kinab:
- ❌ Manuell planering i Excel-listor
- ❌ Telefonsamtal för att avisera kunder
- ❌ Papperslistor för utförare
- ❌ Glömda uppgifter och förlorade intäkter
- ❌ Ingen dokumentation av utförda jobb
- ❌ Ineffektiva rutter och onödiga körkilometer
- ❌ Inkonsistent namngivning och datakaos

Med Unicorn:
- ✅ Automatisk uppgiftsgenerering från klusterstruktur
- ✅ Automatiska SMS-aviseringar

- ✅ Digital mobilapp för utförare
- ✅ Obligatorisk fotografering för dokumentation
- ✅ AI-optimerade rutter
- ✅ Centraliserad data med standardiserad struktur
- ✅ Integration med ekonomisystem för automatisk fakturering

1.2 Systemets Huvudkomponenter

1. Klustret - Hierarkisk Datastruktur

Vad: Ett träd som organiserar både organisatoriska enheter och fysiska objekt

Två objekttyper:
- 🔵 **Organisatoriska** (immateriella): Koncerner, kedjor, distrikt, butiker, områden
- 🟢 **Fysiska** (materiella): Soprum, kärl, maskiner, golvbrunnar, dörrar

Exempel-struktur:

```
▼ 🔵 Axfood (Koncern)
  ▼ 🔵 Willys (Kedja)
    ▼ 🔵 Stockholm (Distrikt)
      ▼ 🔵 Willys Södermalm (Butik)
        ▼ 🔵 Pantrum (Område)
          🟢 Pantmaskin (1 st)
          🟢 Pantkärl 160L (10 st)
          🟢 Golvbrunn (1 st)
```

2. Metadata - “Gula Post-it-lappar”

Vad: Information som hänger på klusterobjekt

Två kategorier:
- **Logisk metadata**: Används i systemets logik (typ, antal, pris, GPS, koder)
- **Ologisk metadata**: Bara information (bilder, fritext, anteckningar)

Nedärvning: Metadata “faller nedåt” från förälder till barn (om inte stoppad)

Exempel:

```
// Metadata på Pantrum
{
  "adress": "Götgatan 100",           // Logisk, ärvs nedåt
  "kod": "1234",                      // Logisk, ärvs nedåt
  "what3words": "index.home.raft",    // Logisk, ärvs inte (exakt position)
  "anteckning": "Trång dörröppning"   // Ologisk, ärvs inte
}

// Metadata på Pantkärl (ärver från Pantrum)
{
  "typ": "Pantkärl",                  // Logisk, ärvs EJ (låst här)
  "storlek": "160L",                  // Logisk, ärvs EJ (låst här)
  "antal": 10,                        // Logisk, ärvs EJ (låst här)
  "adress": "Götgatan 100",           // ← ÄRVT från Pantrum
  "kod": "1234"                       // ← ÄRVT från Pantrum
}
```

3. Orderkoncept - Intelligenta Arbetsordergeneratorer

Vad: En konfigurerad “sökning” som matchar klusterobjekt och skapar uppgifter

Hur det fungerar:
1. Orderkoncept “pekas in” på ett kluster (t.ex. Axfood → Willys)
2. Orderkoncept har filter: `typ = "Pantkärl"`
3. Systemet söker i klustret efter alla objekt som matchar
4. För varje matchning skapas en uppgift
5. **Korsbefruktning**: Antal från metadata multipliceras med artikelpris/tid

Exempel:

```
Orderkoncept: "Tvätta pantkärl"
Filter: typ = "Pantkärl"
Artikel: "Tvätt pantkärl" (50 kr/st, 3 min/st)

Matchning:
- Willys Södermalm → 10 pantkärl
- Willys Vasastan → 15 pantkärl
- Willys Bromma → 8 pantkärl

Resultat:
- Uppgift 1: "Tvätta pantkärl - Willys Södermalm" (10 st, 500 kr, 30 min)
- Uppgift 2: "Tvätta pantkärl - Willys Vasastan" (15 st, 750 kr, 45 min)
- Uppgift 3: "Tvätta pantkärl - Willys Bromma" (8 st, 400 kr, 24 min)
```

4. Artiklar - Tjänster, Varor och Strukturer

Tre artikeltyper:

A. Tjänsteartiklar (det som utförs):
- Tre vikter: Försäljningspris, Estimerad kostnad, Produktionstid
- Exempel: “Tvätta pantkärl” (50 kr, 25 kr, 3 min)

B. Varuartiklar (det som installeras/byts):
- Tre vikter: Försäljningspris, Inköpskostnad, Ledtid
- Exempel: “Lock 240L” (150 kr, 80 kr, 7 dagar)

C. Strukturartiklar (sammansatta arbeten):
- Består av flera underartiklar med beroenden
- Exempel: “Tvätt av soprum” = Etablering + Kodverifiering + Foto före + Tvätt kärl + Tvätt golv + Foto efter + Avetablering

Beroendeuppgifter:
- **före**: Måste göras innan huvuduppgift (t.ex. “Foto före”)
- **efter**: Måste göras efter huvuduppgift (t.ex. “Foto efter”)
- **parallel**: Kan göras samtidigt

5. Snöret - Geografisk och Temporal Planering

Vad: En optimerad rutt för en utförare för en vecka

Består av:
- **Uppgifter**: Jobb som ska utföras

- **Resor**: Körning mellan platser
- **Nattvilor**: Övernattning (vid långresor)

Optimering: Systemet sorterar uppgifter geografiskt för att minimera körsträcka

Veckoplanering: Utförare kan se sin hela vecka, justera preferenser (hotell, arbetstider)

6. Tre Användargrupper

1. Kunder/Brukare:
- Kan se sin klusterstruktur
- Kan felanmäla
- Kan uppdatera metadata (t.ex. koder)

2. Administratörer:
- Bygger klusterstruktur
- Skapar orderkoncept
- Planerar snöret
- Ser rapporter och lönsamhet

3. Utförare:
- Mobilapp med dagens uppgifter
- Tar foton före/efter
- Uppdaterar metadata i fält (adresser, tider, anteckningar)
- Lägger till tillägg (extra tid/artiklar)

1.3 Mats Vision - Sammanfattning

Mats från Kinab har en tydlig vision baserad på 20+ års erfarenhet i branschen:

Kärnprinciper

“Klustret är enkelt, metadatan är komplex”
- Fokusera på ren, enkel klusterstruktur
- All komplexitet ligger i hur metadata hanteras och ärvs

“Efterlikna kundens organisation”
- Klustret ska kännas igen för kunden
- Använd kundens terminologi (Willys säger “pantrum”, använd det)

“Automatisering + mänsklig input”
- Systemet föreslår (AI-guidning, optimering)
- Människan beslutar (godkänner, justerar)

“Dokumentation är affärskritiskt”
- Foton före/efter är INTE optional
- Har förlorat pengar på att inte kunna bevisa utfört arbete
- Försäkringsärenden kräver foto

“Data måste vara åtkomlig i fält”
- Utförare måste kunna uppdatera metadata direkt
- Eliminera papperslistor och manuella anteckningar
- Bättre data = bättre planering nästa gång

Konkreta Exempel från Kinab

Axfood Willys:
- 250 butiker i Sverige
- Varje butik har pantrum, avfallsrum, ibland kundvagnsgarage
- Standardiserat (men små variationer mellan butiker)
- Avisering på TRE nivåer: 3 månader, 3 dagar, 1 timme innan

Stena Fastigheters Sessan:
- Sveriges största bostadskomplex
- 30 identiska soprum
- Perfekt för nedärvning av metadata

Mall of Scandinavia:
- Behov av 3D-navigation (olika våningar)
- GPS fungerar inte inomhus
- Framtida utmaning

1.4 Befintligt System

Nuläge: Kinab använder i dagsläget:
- Excel-listor för kunddata
- Manuell planering
- Telefonsamtal för avisering
- Pappersdokumentation
- Ett äldre system (Modus) som inte uppfyller behoven

Vad som finns idag:
- Kundlistor i Excel med adresser, kontakter
- Prislistor i ekonomisystem (Fortnox)
- Artikelregister i ekonomisystem
- Fotografier sparade lokalt på telefoner (inte systematiskt)

Vad som saknas:
- Klusterstruktur
- Automatisk uppgiftsgenerering
- Rutt-optimering
- Mobilapp för utförare
- Systematisk fotografering
- Metadata-hantering
- Integration mellan system

---

2. Fas 1 - Detaljerad Implementation

2.1 Översikt Fas 1

Mål: Skapa ett fungerande MVP som kan hantera Kinabs kärnverksamhet för EN kund (Axfood Willys) med 3-5 butiker.

Tidsram: 8 veckor

Leveranser:
1. Klusterhantering (CRUD + visualisering)
2. Artikelregister
3. Orderkoncept (sökning + korsbefruktning)
4. Uppgiftshantering (inklusive fotografering)
5. Snöret (grundläggande geografisk sortering)
6. Användarhantering (tre roller)

Teknologier:
- **Frontend**: React 18+ med TypeScript, Tailwind CSS, shadcn/ui
- **Backend**: Next.js 14+ med App Router
- **Databas**: PostgreSQL 15+
- **ORM**: Drizzle ORM
- **Validering**: Zod

2.2 Funktion 1: Datamodell och Databastabeller

A. Funktionsbeskrivning

Vad: Skapa komplett databasschema för alla tabeller som behövs i Fas 1

Varför: Datamodellen är grunden - allt annat bygger på den

Hur: Använd Drizzle ORM för att definiera schema i TypeScript

Tabeller som behövs:
1. `users` - Användare (kunder, admins, utförare)
2. `customers` - Kunder (Axfood, Kinab, etc.)
3. `cluster_objects` - Klusterobjekt (trädet)
4. `metadata` - Metadata på klusterobjekt
5. `articles` - Artiklar (tjänster, varor, strukturer)
6. `article_dependencies` - Beroenden mellan artiklar
7. `price_lists` - Prislistor (sidoregister)
8. `price_list_items` - Artikelpriser i prislistor
9. `order_concepts` - Orderkoncept
10. `concept_filters` - Filter i orderkoncept
11. `assignments` - Uppgifter (genererade från orderkoncept)
12. `assignment_articles` - Artiklar i uppgifter
13. `assignment_extras` - Tillägg (utförare lägger till)
14. `photos` - Foton (före/efter)
15. `ropes` - Snöret (veckoplanering)
16. `rope_segments` - Segment i snöret (uppgift/resa/nattvila)

B. Datamodeller - Drizzle Schema

Fil: `src/db/schema.ts`

```
import { pgTable, uuid, varchar, text, integer, decimal, timestamp, boolean, jsonb,
pgEnum } from 'drizzle-orm/pg-core';
import { relations } from 'drizzle-orm';

// ============================================================
// ENUMS
// ============================================================

export const userRoleEnum = pgEnum('user_role', ['customer', 'admin', 'executor']);
export const clusterTypeEnum = pgEnum('cluster_type', ['organizational', 'physical']);
export const metadataCategoryEnum = pgEnum('metadata_category', ['logical', 'non_logic
al']);
export const articleTypeEnum = pgEnum('article_type', ['service', 'product', 'struc-
ture']);
export const dependencyTypeEnum = pgEnum('dependency_type', ['before', 'after', 'par-
allel']);
export const assignmentStatusEnum = pgEnum('assignment_status', [
  'not_planned',
  'rough_planned',
  'fine_planned',
  'on_way',
  'on_site',
  'completed',
  'ready_for_invoicing',
  'invoiced'
]);
export const photoTypeEnum = pgEnum('photo_type', ['before', 'after']);
export const ropeStatusEnum = pgEnum('rope_status', ['draft', 'confirmed',
'completed']);
export const segmentTypeEnum = pgEnum('segment_type', ['assignment', 'travel', 'night_
rest']);

// ============================================================
// USERS
// ============================================================

export const users = pgTable('users', {
  id: uuid('id').defaultRandom().primaryKey(),
  email: varchar('email', { length: 255 }).notNull().unique(),
  passwordHash: varchar('password_hash', { length: 255 }).notNull(),
  firstName: varchar('first_name', { length: 100 }),
  lastName: varchar('last_name', { length: 100 }),
  phone: varchar('phone', { length: 50 }),
  role: userRoleEnum('role').notNull(),
  customerId: uuid('customer_id').references(() => customers.id),
  active: boolean('active').default(true).notNull(),
  preferences: jsonb('preferences'), // Personliga preferenser (hotell, arbetstider,
etc.)
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// ============================================================
// CUSTOMERS
// ============================================================

export const customers = pgTable('customers', {
  id: uuid('id').defaultRandom().primaryKey(),
  name: varchar('name', { length: 255 }).notNull(),
  organizationNumber: varchar('organization_number', { length: 50 }),
  contactEmail: varchar('contact_email', { length: 255 }),
  contactPhone: varchar('contact_phone', { length: 50 }),
```

```
  address: text('address'),
  notes: text('notes'),
  active: boolean('active').default(true).notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// ==============================================================
// CLUSTER OBJECTS (Hierarkiskt träd)
// ==============================================================

export const clusterObjects = pgTable('cluster_objects', {
  id: uuid('id').defaultRandom().primaryKey(),
  parentId: uuid('parent_id').references(() => clusterObjects.id, { onDelete: 'cas-
cade' }),
  customerId: uuid('customer_id').references(() => customers.id).notNull(),
  name: varchar('name', { length: 255 }).notNull(),
  type: clusterTypeEnum('type').notNull(),
  level: integer('level').notNull(), // 1 = topp, 2 = division, 3 = geografiskt, etc.
  description: text('description'),
  structureInfo: jsonb('structure_info'), // Flexibel data för olika typer
  sortOrder: integer('sort_order').default(0), // För att sortera syskon
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// Index för effektiv träd-traversering
// CREATE INDEX idx_cluster_parent ON cluster_objects(parent_id);
// CREATE INDEX idx_cluster_customer ON cluster_objects(customer_id);
// CREATE INDEX idx_cluster_type ON cluster_objects(type);

// ==============================================================
// METADATA (Gula post-it-lappar)
// ==============================================================

export const metadata = pgTable('metadata', {
  id: uuid('id').defaultRandom().primaryKey(),
  clusterObjectId: uuid('cluster_object_id').references(() => clusterObjects.id, { onD
elete: 'cascade' }).notNull(),
  key: varchar('key', { length: 100 }).notNull(), // t.ex. "typ", "antal", "adress",
"kod"
  value: jsonb('value').notNull(), // Flexibel value (string, number, array, etc.)
  category: metadataCategoryEnum('category').notNull(),
  inheritable: boolean('inheritable').default(true).notNull(), // Ska metadatan ärvas
nedåt?
  inheritedFrom: uuid('inherited_from').references(() => clusterObjects.id), // Om
ärvd, varifrån?
  dataType: varchar('data_type', { length: 50 }), // 'string', 'number', 'boolean',
'array', 'object'
  unit: varchar('unit', { length: 50 }), // t.ex. 'kr', 'min', 'kvm', 'st'
  description: text('description'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// Index för snabb metadata-sökning (KRITISKT för orderkoncept!)
// CREATE INDEX idx_metadata_object ON metadata(cluster_object_id);
// CREATE INDEX idx_metadata_key ON metadata(key);
// CREATE INDEX idx_metadata_inheritable ON metadata(inheritable);
// CREATE INDEX idx_metadata_category ON metadata(category);
// CREATE INDEX idx_metadata_key_value ON metadata(key, value) USING GIN; -- För
JSONB-sökning
```

```
// ============================================================
// ARTICLES (Artikelregister - sidoregister)
// ============================================================

export const articles = pgTable('articles', {
  id: uuid('id').defaultRandom().primaryKey(),
  code: varchar('code', { length: 100 }).notNull().unique(), // Artikelkod (matchar
ekonomisystem)
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  type: articleTypeEnum('type').notNull(),

  // Tre vikter
  weight1: decimal('weight1', { precision: 10, scale: 2 }), // Tjänst: Försäljnings-
pris, Vara: Försäljningspris
  weight2: decimal('weight2', { precision: 10, scale: 2 }), // Tjänst: Kostnad, Vara:
Inköpskostnad
  weight3: decimal('weight3', { precision: 10, scale: 2 }), // Tjänst: Produktionstid
(min), Vara: Ledtid (dagar)

  // Labels för vikterna (flexibelt)
  weight1Label: varchar('weight1_label', { length: 50 }).default('Pris'),
  weight2Label: varchar('weight2_label', { length: 50 }).default('Kostnad'),
  weight3Label: varchar('weight3_label', { length: 50 }).default('Tid'),

  // Konfigurera multiplikation
  multiplyByMetadata: varchar('multiply_by_metadata', { length: 100 }), // t.ex. "ant-
al", "kvm"
  lockToAddress:
boolean('lock_to_address').default(false), // Etablering/Avetablering ska inte
multipliceras per objekt

  // Struktur (för strukturartiklar)
  isStructure: boolean('is_structure').default(false),

  active: boolean('active').default(true).notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// ============================================================
// ARTICLE_DEPENDENCIES (Beroendeuppgifter)
// ============================================================

export const articleDependencies = pgTable('article_dependencies', {
  id: uuid('id').defaultRandom().primaryKey(),
  parentArticleId: uuid('parent_article_id').references(() => articles.id, {
onDelete: 'cascade' }).notNull(),
  childArticleId: uuid('child_article_id').references(() => articles.id, { onDelete: '
cascade' }).notNull(),
  dependencyType: dependencyTypeEnum('dependency_type').notNull(),
  orderIndex: integer('order_index').notNull(), // Ordning i strukturen
  required: boolean('required').default(true), // Måste den göras?
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// ============================================================
// PRICE LISTS (Sidoregister)
// ============================================================

export const priceLists = pgTable('price_lists', {
  id: uuid('id').defaultRandom().primaryKey(),
  customerId: uuid('customer_id').references(() => customers.id),
```

```
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  validFrom: timestamp('valid_from'),
  validTo: timestamp('valid_to'),
  isDefault: boolean('is_default').default(false),
  active: boolean('active').default(true).notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

export const priceListItems = pgTable('price_list_items', {
  id: uuid('id').defaultRandom().primaryKey(),
  priceListId: uuid('price_list_id').references(() => priceLists.id, { onDelete: 'cas-
cade' }).notNull(),
  articleId: uuid('article_id').references(() => articles.id).notNull(),
  price: decimal('price', { precision: 10, scale: 2 }).notNull(),
  cost: decimal('cost', { precision: 10, scale: 2 }),
  timeOrLeadtime: decimal('time_or_leadtime', { precision: 10, scale: 2 }),
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// ============================================================
// ORDER CONCEPTS (Intelligenta arbetsordergeneratorer)
// ============================================================

export const orderConcepts = pgTable('order_concepts', {
  id: uuid('id').defaultRandom().primaryKey(),
  customerId: uuid('customer_id').references(() => customers.id).notNull(),
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  targetClusterId: uuid('target_cluster_id').references(() => clusterObjects.id), //
Var i klustret söker den?
  articleId: uuid('article_id').references(() => articles.id), // Vilken artikel ska-
pas uppgifter med?
  configuration: jsonb('configuration'), // Flexibel konfiguration
  active: boolean('active').default(true).notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

export const conceptFilters = pgTable('concept_filters', {
  id: uuid('id').defaultRandom().primaryKey(),
  orderConceptId: uuid('order_concept_id').references(() => orderConcepts.id, { onDele
te: 'cascade' }).notNull(),
  metadataKey: varchar('metadata_key', { length: 100 }).notNull(), // t.ex. "typ"
  operator: varchar('operator', { length: 50 }).notNull(), // 'equals', 'contains',
'greater_than', etc.
  value: jsonb('value').notNull(), // Värdet att matcha mot
  priority: integer('priority').default(0), // För AND/OR-logik i framtiden
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// ============================================================
// ASSIGNMENTS (Uppgifter)
// ============================================================

export const assignments = pgTable('assignments', {
  id: uuid('id').defaultRandom().primaryKey(),
  orderConceptId: uuid('order_concept_id').references(() => orderConcepts.id),
  clusterObjectId: uuid('cluster_object_id').references(() => clusterOb-
jects.id).notNull(),
  customerId: uuid('customer_id').references(() => customers.id).notNull(),
  executorId: uuid('executor_id').references(() => users.id),
```

```
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  status: assignmentStatusEnum('status').default('not_planned').notNull(),

  // Tidplanering
  scheduledDate: timestamp('scheduled_date'),
  scheduledTimeStart: varchar('scheduled_time_start', { length: 10 }), // "09:00"
  scheduledTimeEnd: varchar('scheduled_time_end', { length: 10 }), // "10:30"
  actualStartTime: timestamp('actual_start_time'),
  actualEndTime: timestamp('actual_end_time'),

  // Kundönskemål
  customerPreferredDate: timestamp('customer_preferred_date'),
  customerTimeWindow: varchar('customer_time_window', { length: 100 }), // "Kl
06:00-10:00"
  customerNotes: text('customer_notes'),

  // Metadata-snapshot (vid uppgiftsskapande)
  metadataSnapshot: jsonb('metadata_snapshot'), // All relevant metadata vid skapandet

  // Beräknade värden
  estimatedPrice: decimal('estimated_price', { precision: 10, scale: 2 }),
  estimatedCost: decimal('estimated_cost', { precision: 10, scale: 2 }),
  estimatedTime: decimal('estimated_time', { precision: 10, scale: 2 }), // minuter

  // Faktiska värden (efterkalkylering)
  actualPrice: decimal('actual_price', { precision: 10, scale: 2 }),
  actualCost: decimal('actual_cost', { precision: 10, scale: 2 }),
  actualTime: decimal('actual_time', { precision: 10, scale: 2 }), // minuter

  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// Index för snabb filtrering
// CREATE INDEX idx_assignment_status ON assignments(status);
// CREATE INDEX idx_assignment_date ON assignments(scheduled_date);
// CREATE INDEX idx_assignment_executor ON assignments(executor_id);
// CREATE INDEX idx_assignment_customer ON assignments(customer_id);
// CREATE INDEX idx_assignment_cluster ON assignments(cluster_object_id);

// ============================================================
// ASSIGNMENT_ARTICLES (Artiklar i uppgifter)
// ============================================================

export const assignmentArticles = pgTable('assignment_articles', {
  id: uuid('id').defaultRandom().primaryKey(),
  assignmentId: uuid('assignment_id').references(() => assignments.id, { onDelete: 'ca
scade' }).notNull(),
  articleId: uuid('article_id').references(() => articles.id).notNull(),
  quantity: decimal('quantity', { precision: 10, scale: 2 }).default('1').notNull(),

  // Beräknade värden (vid skapande)
  calculatedPrice: decimal('calculated_price', { precision: 10, scale: 2 }),
  calculatedCost: decimal('calculated_cost', { precision: 10, scale: 2 }),
  calculatedTime: decimal('calculated_time', { precision: 10, scale: 2 }),

  orderIndex: integer('order_index'), // Ordning i strukturen
  completed: boolean('completed').default(false),
  completedAt: timestamp('completed_at'),

  createdAt: timestamp('created_at').defaultNow().notNull(),
```

```
});

// ============================================================
// ASSIGNMENT_EXTRAS (Tillägg från utförare)
// ============================================================

export const assignmentExtras = pgTable('assignment_extras', {
  id: uuid('id').defaultRandom().primaryKey(),
  assignmentId: uuid('assignment_id').references(() => assignments.id, { onDelete: 'ca
scade' }).notNull(),
  addedBy: uuid('added_by').references(() => users.id).notNull(),

  type: varchar('type', { length: 50 }).notNull(), // 'article', 'time', 'note'
  articleId: uuid('article_id').references(() => articles.id),
  quantity: decimal('quantity', { precision: 10, scale: 2 }),
  extraTime: decimal('extra_time', { precision: 10, scale: 2 }), // minuter
  reason: text('reason').notNull(), // Motivering KRÄVS

  price: decimal('price', { precision: 10, scale: 2 }),
  cost: decimal('cost', { precision: 10, scale: 2 }),

  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// ============================================================
// PHOTOS (Dokumentation)
// ============================================================

export const photos = pgTable('photos', {
  id: uuid('id').defaultRandom().primaryKey(),
  assignmentId: uuid('assignment_id').references(() => assignments.id, { onDelete: 'ca
scade' }).notNull(),
  uploadedBy: uuid('uploaded_by').references(() => users.id).notNull(),
  type: photoTypeEnum('type').notNull(),

  url: text('url').notNull(), // Filsökväg eller URL
  thumbnailUrl: text('thumbnail_url'),
  fileSize: integer('file_size'), // bytes
  mimeType: varchar('mime_type', { length: 100 }),

  // Metadata från foto (EXIF)
  photoMetadata: jsonb('photo_metadata'), // GPS, timestamp, kamera, etc.

  notes: text('notes'),
  takenAt: timestamp('taken_at').notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// CREATE INDEX idx_photo_assignment ON photos(assignment_id);
// CREATE INDEX idx_photo_type ON photos(type);

// ============================================================
// ROPES (Snöret - veckoplanering)
// ============================================================

export const ropes = pgTable('ropes', {
  id: uuid('id').defaultRandom().primaryKey(),
  executorId: uuid('executor_id').references(() => users.id).notNull(),
  name: varchar('name', { length: 255 }),

  weekStart: timestamp('week_start').notNull(),
  weekEnd: timestamp('week_end').notNull(),
```

```
  status: ropeStatusEnum('status').default('draft').notNull(),

  // Beräknade värden
  totalDistance: decimal('total_distance', { precision: 10, scale: 2 }), // km
  totalDriveTime: decimal('total_drive_time', { precision: 10, scale: 2 }), // minuter
  totalWorkTime: decimal('total_work_time', { precision: 10, scale: 2 }), // minuter

  optimizedAt: timestamp('optimized_at'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// CREATE INDEX idx_rope_executor_week ON ropes(executor_id, week_start);

// ============================================================
// ROPE_SEGMENTS (Segment i snöret)
// ============================================================

export const ropeSegments = pgTable('rope_segments', {
  id: uuid('id').defaultRandom().primaryKey(),
  ropeId: uuid('rope_id').references(() => ropes.id, { onDelete: 'cascade' }).not-
Null(),

  type: segmentTypeEnum('type').notNull(),
  orderIndex: integer('order_index').notNull(), // Ordning i snöret

  // Om type = 'assignment'
  assignmentId: uuid('assignment_id').references(() => assignments.id),

  // Om type = 'travel'
  fromLat: decimal('from_lat', { precision: 10, scale: 7 }),
  fromLng: decimal('from_lng', { precision: 10, scale: 7 }),
  toLat: decimal('to_lat', { precision: 10, scale: 7 }),
  toLng: decimal('to_lng', { precision: 10, scale: 7 }),
  distance: decimal('distance', { precision: 10, scale: 2 }), // km
  duration: decimal('duration', { precision: 10, scale: 2 }), // minuter

  // Om type = 'night_rest'
  hotelName: varchar('hotel_name', { length: 255 }),
  hotelAddress: text('hotel_address'),

  // Tidplanering
  startTime: timestamp('start_time'),
  endTime: timestamp('end_time'),

  routeInfo: jsonb('route_info'), // Flexibel data från Google Maps

  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// CREATE INDEX idx_rope_segment_order ON rope_segments(rope_id, order_index);

// ============================================================
// RELATIONS (Drizzle ORM Relations)
// ============================================================

export const usersRelations = relations(users, ({ one, many }) => ({
  customer: one(customers, {
    fields: [users.customerId],
    references: [customers.id],
  }),
  assignmentsAsExecutor: many(assignments),
  ropesAsExecutor: many(ropes),
```

```
}));

export const customersRelations = relations(customers, ({ many }) => ({
  users: many(users),
  clusterObjects: many(clusterObjects),
  priceLists: many(priceLists),
  orderConcepts: many(orderConcepts),
  assignments: many(assignments),
}));

export const clusterObjectsRelations = relations(clusterObjects, ({ one, many }) => ({
  parent: one(clusterObjects, {
    fields: [clusterObjects.parentId],
    references: [clusterObjects.id],
  }),
  children: many(clusterObjects),
  customer: one(customers, {
    fields: [clusterObjects.customerId],
    references: [customers.id],
  }),
  metadata: many(metadata),
  assignments: many(assignments),
}));

export const metadataRelations = relations(metadata, ({ one }) => ({
  clusterObject: one(clusterObjects, {
    fields: [metadata.clusterObjectId],
    references: [clusterObjects.id],
  }),
  inheritedFromObject: one(clusterObjects, {
    fields: [metadata.inheritedFrom],
    references: [clusterObjects.id],
  }),
}));

export const articlesRelations = relations(articles, ({ many }) => ({
  dependenciesAsParent: many(articleDependencies, { relationName: 'parentArticle' }),
  dependenciesAsChild: many(articleDependencies, { relationName: 'childArticle' }),
  priceListItems: many(priceListItems),
  orderConcepts: many(orderConcepts),
  assignmentArticles: many(assignmentArticles),
}));

export const orderConceptsRelations = relations(orderConcepts, ({ one, many }) => ({
  customer: one(customers, {
    fields: [orderConcepts.customerId],
    references: [customers.id],
  }),
  targetCluster: one(clusterObjects, {
    fields: [orderConcepts.targetClusterId],
    references: [clusterObjects.id],
  }),
  article: one(articles, {
    fields: [orderConcepts.articleId],
    references: [articles.id],
  }),
  filters: many(conceptFilters),
  assignments: many(assignments),
}));

export const assignmentsRelations = relations(assignments, ({ one, many }) => ({
  orderConcept: one(orderConcepts, {
    fields: [assignments.orderConceptId],
```

```
    references: [orderConcepts.id],
  }),
  clusterObject: one(clusterObjects, {
    fields: [assignments.clusterObjectId],
    references: [clusterObjects.id],
  }),
  customer: one(customers, {
    fields: [assignments.customerId],
    references: [customers.id],
  }),
  executor: one(users, {
    fields: [assignments.executorId],
    references: [users.id],
  }),
  articles: many(assignmentArticles),
  extras: many(assignmentExtras),
  photos: many(photos),
  ropeSegments: many(ropeSegments),
}));

export const ropesRelations = relations(ropes, ({ one, many }) => ({
  executor: one(users, {
    fields: [ropes.executorId],
    references: [users.id],
  }),
  segments: many(ropeSegments),
}));

export const ropeSegmentsRelations = relations(ropeSegments, ({ one }) => ({
  rope: one(ropes, {
    fields: [ropeSegments.ropeId],
    references: [ropes.id],
  }),
  assignment: one(assignments, {
    fields: [ropeSegments.assignmentId],
    references: [assignments.id],
  }),
}));
```

C. Backend Implementation

Fil: `src/db/index.ts` (Database connection)

```
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool } from 'pg';
import * as schema from './schema';

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

export const db = drizzle(pool, { schema });
```

Fil: `drizzle.config.ts` (Drizzle Kit config för migrations)

```typescript
import type { Config } from 'drizzle-kit';

export default {
  schema: './src/db/schema.ts',
  out: './drizzle',
  driver: 'pg',
  dbCredentials: {
    connectionString: process.env.DATABASE_URL!,
  },
} satisfies Config;
```

Migrations-kommandon:

```bash
# Generera migration från schema
npx drizzle-kit generate:pg

# Kör migrations
npx drizzle-kit push:pg

# Öppna Drizzle Studio för att se data
npx drizzle-kit studio
```

D. Frontend Implementation

(Inget UI behövs för datamodellen i sig, men Drizzle Studio kan användas för debugging)

E. Testning

Unit Test: `src/db/__tests__/schema.test.ts`

```
import { describe, it, expect } from '@jest/globals';
import { db } from '../index';
import { customers, clusterObjects, metadata } from '../schema';
import { eq } from 'drizzle-orm';

describe('Database Schema', () => {
  it('should create customer', async () => {
    const [customer] = await db.insert(customers).values({
      name: 'Test Customer AB',
      organizationNumber: '556677-8899',
      active: true,
    }).returning();

    expect(customer.id).toBeDefined();
    expect(customer.name).toBe('Test Customer AB');
  });

  it('should create cluster hierarchy', async () => {
    // Skapa kund
    const [customer] = await db.insert(customers).values({
      name: 'Axfood',
    }).returning();

    // Skapa toppnivå
    const [topLevel] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      name: 'Axfood Koncern',
      type: 'organizational',
      level: 1,
    }).returning();

    // Skapa barn
    const [child] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      parentId: topLevel.id,
      name: 'Willys',
      type: 'organizational',
      level: 2,
    }).returning();

    expect(child.parentId).toBe(topLevel.id);
  });

  it('should add metadata to cluster object', async () => {
    // Skapa kund och objekt
    const [customer] = await db.insert(customers).values({ name: 'Test' }).return-
ing();
    const [obj] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      name: 'Pantrum',
      type: 'organizational',
      level: 3,
    }).returning();

    // Lägg till metadata
    const [meta] = await db.insert(metadata).values({
      clusterObjectId: obj.id,
      key: 'adress',
      value: '"Götgatan 100"',
      category: 'logical',
      inheritable: true,
    }).returning();
```

```
    expect(meta.key).toBe('adress');
    expect(meta.inheritable).toBe(true);
  });
});
```

F. Acceptanskriterier

- [ ] Alla tabeller skapas korrekt i PostgreSQL
- [ ] Foreign keys fungerar (cascade delete testat)
- [ ] Index skapas för snabb sökning
- [ ] Drizzle Studio visar alla tabeller
- [ ] Test-data kan skapas och läsas
- [ ] Migrations kan köras upp och ner utan fel

2.3 Funktion 2: API - CRUD Klusterobjekt

A. Funktionsbeskrivning

Vad: REST API endpoints för att skapa, läsa, uppdatera och ta bort klusterobjekt

Varför: Administratörer måste kunna bygga klusterstrukturen via GUI

Hur: Next.js API routes med Drizzle ORM

Endpoints:
- `GET /api/clusters` - Lista alla (med pagination och filtrering)
- `POST /api/clusters` - Skapa nytt objekt
- `GET /api/clusters/:id` - Hämta specifikt objekt
- `PUT /api/clusters/:id` - Uppdatera objekt
- `DELETE /api/clusters/:id` - Ta bort objekt (och alla barn)
- `GET /api/clusters/:id/tree` - Hämta hela trädet från denna nod
- `GET /api/clusters/:id/metadata` - Hämta all metadata (ärvd + lokal)

B. Backend Implementation

Fil: `src/app/api/clusters/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@/db';
import { clusterObjects } from '@/db/schema';
import { eq, and, isNull } from 'drizzle-orm';
import { z } from 'zod';

// ============================================================
// VALIDATION SCHEMAS
// ============================================================

const createClusterObjectSchema = z.object({
  parentId: z.string().uuid().nullable(),
  customerId: z.string().uuid(),
  name: z.string().min(1).max(255),
  type: z.enum(['organizational', 'physical']),
  level: z.number().int().min(1),
  description: z.string().optional(),
  structureInfo: z.any().optional(),
  sortOrder: z.number().int().default(0),
});

// ============================================================
// GET /api/clusters - Lista alla kluster
// ============================================================

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);

    // Filtrering
    const customerId = searchParams.get('customerId');
    const parentId = searchParams.get('parentId');
    const type = searchParams.get('type');
    const topLevelOnly = searchParams.get('topLevelOnly') === 'true';

    // Pagination
    const page = parseInt(searchParams.get('page') || '1');
    const pageSize = parseInt(searchParams.get('pageSize') || '50');
    const offset = (page - 1) * pageSize;

    // Bygg query
    const conditions = [];
    if (customerId) {
      conditions.push(eq(clusterObjects.customerId, customerId));
    }
    if (parentId) {
      conditions.push(eq(clusterObjects.parentId, parentId));
    }
    if (topLevelOnly) {
      conditions.push(isNull(clusterObjects.parentId));
    }
    if (type) {
      conditions.push(eq(clusterObjects.type, type as any));
    }

    const result = await db.query.clusterObjects.findMany({
      where: conditions.length > 0 ? and(...conditions) : undefined,
      limit: pageSize,
      offset: offset,
      orderBy: (clusterObjects, { asc }) => [asc(clusterObjects.sortOrder), asc(cluste
rObjects.name)],
    });
```

```
    return NextResponse.json({
      success: true,
      data: result,
      meta: {
        page,
        pageSize,
        total: result.length, // I produktion: gör en separat COUNT-query
      },
    });
  } catch (error) {
    console.error('GET /api/clusters error:', error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to fetch cluster objects',
        },
      },
      { status: 500 }
    );
  }
}

// ============================================================
// POST /api/clusters - Skapa nytt klusterobjekt
// ============================================================

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();

    // Validera input
    const validation = createClusterObjectSchema.safeParse(body);
    if (!validation.success) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'VALIDATION_ERROR',
            message: 'Invalid input',
            details: validation.error.errors,
          },
        },
        { status: 400 }
      );
    }

    const data = validation.data;

    // Om det är barn, verifiera att förälder finns
    if (data.parentId) {
      const parent = await db.query.clusterObjects.findFirst({
        where: eq(clusterObjects.id, data.parentId),
      });

      if (!parent) {
        return NextResponse.json(
          {
            success: false,
            error: {
              code: 'PARENT_NOT_FOUND',
```

```
              message: `Parent cluster object with id ${data.parentId} not found`,
            },
          },
          { status: 404 }
        );
      }

      // Verifiera att level är rätt (barnets level = förälderns level + 1)
      if (data.level !== parent.level + 1) {
        return NextResponse.json(
          {
            success: false,
            error: {
              code: 'INVALID_LEVEL',
              message: `Child level must be parent level + 1 (parent level: ${par-
ent.level})`,
            },
          },
          { status: 400 }
        );
      }
    }

    // Skapa klusterobjekt
    const [newClusterObject] = await db.insert(clusterObjects).values(data).returning(
);

    return NextResponse.json(
      {
        success: true,
        data: newClusterObject,
      },
      { status: 201 }
    );
  } catch (error) {
    console.error('POST /api/clusters error:', error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to create cluster object',
        },
      },
      { status: 500 }
    );
  }
}
```

Fil: `src/app/api/clusters/[id]/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@/db';
import { clusterObjects } from '@/db/schema';
import { eq } from 'drizzle-orm';
import { z } from 'zod';

const updateClusterObjectSchema = z.object({
  name: z.string().min(1).max(255).optional(),
  description: z.string().optional(),
  structureInfo: z.any().optional(),
  sortOrder: z.number().int().optional(),
});

// ============================================================
// GET /api/clusters/:id - Hämta specifikt klusterobjekt
// ============================================================

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const { id } = params;

    const clusterObject = await db.query.clusterObjects.findFirst({
      where: eq(clusterObjects.id, id),
      with: {
        parent: true,
        children: {
          orderBy: (clusterObjects, { asc }) => [asc(clusterObjects.sortOrder)],
        },
        metadata: true,
      },
    });

    if (!clusterObject) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'NOT_FOUND',
            message: `Cluster object with id ${id} not found`,
          },
        },
        { status: 404 }
      );
    }

    return NextResponse.json({
      success: true,
      data: clusterObject,
    });
  } catch (error) {
    console.error(`GET /api/clusters/${params.id} error:`, error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to fetch cluster object',
        },
      },
```

```
      { status: 500 }
    );
  }
}

// ==============================================================
// PUT /api/clusters/:id - Uppdatera klusterobjekt
// ==============================================================

export async function PUT(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const { id } = params;
    const body = await request.json();

    // Validera input
    const validation = updateClusterObjectSchema.safeParse(body);
    if (!validation.success) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'VALIDATION_ERROR',
            message: 'Invalid input',
            details: validation.error.errors,
          },
        },
        { status: 400 }
      );
    }

    const data = validation.data;

    // Uppdatera
    const [updated] = await db
      .update(clusterObjects)
      .set({
        ...data,
        updatedAt: new Date(),
      })
      .where(eq(clusterObjects.id, id))
      .returning();

    if (!updated) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'NOT_FOUND',
            message: `Cluster object with id ${id} not found`,
          },
        },
        { status: 404 }
      );
    }

    return NextResponse.json({
      success: true,
      data: updated,
    });
  } catch (error) {
```

```
    console.error(`PUT /api/clusters/${params.id} error:`, error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to update cluster object',
        },
      },
      { status: 500 }
    );
  }
}

// ============================================================
// DELETE /api/clusters/:id - Ta bort klusterobjekt
// ============================================================

export async function DELETE(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const { id } = params;

    // Kontrollera att objektet finns
    const existing = await db.query.clusterObjects.findFirst({
      where: eq(clusterObjects.id, id),
      with: {
        children: true,
      },
    });

    if (!existing) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'NOT_FOUND',
            message: `Cluster object with id ${id} not found`,
          },
        },
        { status: 404 }
      );
    }

    // Varna om objektet har barn (cascade kommer ta bort dem)
    const childCount = existing.children?.length || 0;

    // Ta bort (cascade kommer ta bort barn och metadata)
    await db.delete(clusterObjects).where(eq(clusterObjects.id, id));

    return NextResponse.json({
      success: true,
      data: {
        deletedId: id,
        deletedChildrenCount: childCount,
        message: childCount > 0
          ? `Deleted cluster object and ${childCount} children`
          : 'Deleted cluster object',
      },
    });
  } catch (error) {
```

```
    console.error(`DELETE /api/clusters/${params.id} error:`, error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to delete cluster object',
        },
      },
      { status: 500 }
    );
  }
}
```

Fil: `src/app/api/clusters/[id]/tree/route.ts` (Hämta hela trädet)

```
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@/db';
import { clusterObjects, metadata } from '@/db/schema';
import { eq } from 'drizzle-orm';

// Rekursiv funktion för att hämta träd
async function fetchTreeRecursive(nodeId: string, depth: number = 0, maxDepth: number
= 10): Promise<any> {
  if (depth > maxDepth) {
    throw new Error('Max tree depth exceeded (possible circular reference)');
  }

  const node = await db.query.clusterObjects.findFirst({
    where: eq(clusterObjects.id, nodeId),
    with: {
      metadata: true,
    },
  });

  if (!node) {
    return null;
  }

  // Hämta alla barn
  const children = await db.query.clusterObjects.findMany({
    where: eq(clusterObjects.parentId, nodeId),
    orderBy: (clusterObjects, { asc }) => [asc(clusterObjects.sortOrder), asc(clusterO
bjects.name)],
  });

  // Rekursivt hämta barnens träd
  const childrenWithTree = await Promise.all(
    children.map(child => fetchTreeRecursive(child.id, depth + 1, maxDepth))
  );

  return {
    ...node,
    children: childrenWithTree,
  };
}

// ============================================================
// GET /api/clusters/:id/tree - Hämta hela trädet från nod
// ============================================================

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const { id } = params;
    const { searchParams } = new URL(request.url);
    const maxDepth = parseInt(searchParams.get('maxDepth') || '10');

    const tree = await fetchTreeRecursive(id, 0, maxDepth);

    if (!tree) {
      return NextResponse.json(
        {
          success: false,
          error: {
            code: 'NOT_FOUND',
```

```
            message: `Cluster object with id ${id} not found`,
          },
        },
        { status: 404 }
      );
    }

    return NextResponse.json({
      success: true,
      data: tree,
    });
  } catch (error) {
    console.error(`GET /api/clusters/${params.id}/tree error:`, error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: error instanceof Error ? error.message : 'Failed to fetch cluster
tree',
        },
      },
      { status: 500 }
    );
  }
}
```

C. Frontend Implementation

Fil: `src/lib/api/clusters.ts` (API client)

```
import { ClusterObject } from '@/types';

export async function getClusters(params?: {
  customerId?: string;
  parentId?: string;
  type?: 'organizational' | 'physical';
  topLevelOnly?: boolean;
  page?: number;
  pageSize?: number;
}) {
  const searchParams = new URLSearchParams();
  if (params) {
    Object.entries(params).forEach(([key, value]) => {
      if (value !== undefined) {
        searchParams.append(key, String(value));
      }
    });
  }

  const response = await fetch(`/api/clusters?${searchParams}`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json' },
  });

  return response.json();
}

export async function getClusterById(id: string) {
  const response = await fetch(`/api/clusters/${id}`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json' },
  });

  return response.json();
}

export async function getClusterTree(id: string, maxDepth?: number) {
  const searchParams = new URLSearchParams();
  if (maxDepth) {
    searchParams.append('maxDepth', String(maxDepth));
  }

  const response = await fetch(`/api/clusters/${id}/tree?${searchParams}`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json' },
  });

  return response.json();
}

export async function createCluster(data: {
  parentId?: string | null;
  customerId: string;
  name: string;
  type: 'organizational' | 'physical';
  level: number;
  description?: string;
  structureInfo?: any;
  sortOrder?: number;
}) {
  const response = await fetch('/api/clusters', {
    method: 'POST',
```

```
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  return response.json();
}

export async function updateCluster(id: string, data: {
  name?: string;
  description?: string;
  structureInfo?: any;
  sortOrder?: number;
}) {
  const response = await fetch(`/api/clusters/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  return response.json();
}

export async function deleteCluster(id: string) {
  const response = await fetch(`/api/clusters/${id}`, {
    method: 'DELETE',
    headers: { 'Content-Type': 'application/json' },
  });

  return response.json();
}
```

Fil: `src/components/ClusterList.tsx` (Enkel lista)

```
'use client';

import { useEffect, useState } from 'react';
import { getClusters } from '@/lib/api/clusters';

interface ClusterObject {
  id: string;
  name: string;
  type: 'organizational' | 'physical';
  level: number;
  parentId?: string | null;
}

export function ClusterList({ customerId }: { customerId: string }) {
  const [clusters, setClusters] = useState<ClusterObject[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    async function fetchClusters() {
      try {
        setLoading(true);
        const response = await getClusters({ customerId, topLevelOnly: true });

        if (response.success) {
          setClusters(response.data);
        } else {
          setError(response.error.message);
        }
      } catch (err) {
        setError('Failed to fetch clusters');
        console.error(err);
      } finally {
        setLoading(false);
      }
    }

    fetchClusters();
  }, [customerId]);

  if (loading) {
    return <div>Loading clusters...</div>;
  }

  if (error) {
    return <div className="text-red-500">Error: {error}</div>;
  }

  if (clusters.length === 0) {
    return <div>No clusters found. Create your first cluster!</div>;
  }

  return (
    <div className="space-y-2">
      <h2 className="text-2xl font-bold">Clusters</h2>
      <ul className="space-y-1">
        {clusters.map((cluster) => (
          <li key={cluster.id} className="p-2 border rounded hover:bg-gray-50">
            <span className={cluster.type === 'organizational' ? 'text-blue-600' : 'te
xt-green-600'}>
              {cluster.type === 'organizational' ? '🔵' : '🟢'}
            </span>{' '}
```

```
            {cluster.name}
            <span className="text-gray-500 text-sm ml-2">(Level {cluster.level})</
span>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

D. Testning

Integration Test: `src/app/api/clusters/__tests__/route.test.ts`

```
import { describe, it, expect, beforeAll, afterAll } from '@jest/globals';
import { db } from '@/db';
import { customers, clusterObjects } from '@/db/schema';
import { POST as createCluster, GET as getClusters } from '../route';
import { GET as getClusterById, DELETE as deleteCluster } from '../[id]/route';

describe('POST /api/clusters', () => {
  let testCustomer: any;

  beforeAll(async () => {
    [testCustomer] = await db.insert(customers).values({
      name: 'Test Customer',
    }).returning();
  });

  afterAll(async () => {
    await db.delete(customers).where(eq(customers.id, testCustomer.id));
  });

  it('should create top-level cluster object', async () => {
    const request = new Request('http://localhost/api/clusters', {
      method: 'POST',
      body: JSON.stringify({
        customerId: testCustomer.id,
        name: 'Axfood',
        type: 'organizational',
        level: 1,
      }),
    });

    const response = await createCluster(request as any);
    const data = await response.json();

    expect(data.success).toBe(true);
    expect(data.data.name).toBe('Axfood');
    expect(data.data.level).toBe(1);
    expect(data.data.parentId).toBeNull();
  });

  it('should create child cluster object', async () => {
    // Skapa förälder
    const [parent] = await db.insert(clusterObjects).values({
      customerId: testCustomer.id,
      name: 'Axfood',
      type: 'organizational',
      level: 1,
    }).returning();

    // Skapa barn
    const request = new Request('http://localhost/api/clusters', {
      method: 'POST',
      body: JSON.stringify({
        customerId: testCustomer.id,
        parentId: parent.id,
        name: 'Willys',
        type: 'organizational',
        level: 2,
      }),
    });

    const response = await createCluster(request as any);
    const data = await response.json();
```

```
    expect(data.success).toBe(true);
    expect(data.data.name).toBe('Willys');
    expect(data.data.parentId).toBe(parent.id);
    expect(data.data.level).toBe(2);
  });

  it('should reject invalid level', async () => {
    const [parent] = await db.insert(clusterObjects).values({
      customerId: testCustomer.id,
      name: 'Axfood',
      type: 'organizational',
      level: 1,
    }).returning();

    const request = new Request('http://localhost/api/clusters', {
      method: 'POST',
      body: JSON.stringify({
        customerId: testCustomer.id,
        parentId: parent.id,
        name: 'Willys',
        type: 'organizational',
        level: 5, // FELAKTIGT! Borde vara 2
      }),
    });

    const response = await createCluster(request as any);
    const data = await response.json();

    expect(data.success).toBe(false);
    expect(data.error.code).toBe('INVALID_LEVEL');
  });
});

describe('GET /api/clusters', () => {
  // ... liknande tester
});

describe('DELETE /api/clusters/:id', () => {
  it('should delete cluster and all children (cascade)', async () => {
    // Skapa hierarki: Axfood → Willys → Willys Södermalm
    const [customer] = await db.insert(customers).values({ name: 'Test' }).return-
ing();
    const [axfood] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      name: 'Axfood',
      type: 'organizational',
      level: 1,
    }).returning();
    const [willys] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      parentId: axfood.id,
      name: 'Willys',
      type: 'organizational',
      level: 2,
    }).returning();
    const [butik] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      parentId: willys.id,
      name: 'Willys Södermalm',
      type: 'organizational',
      level: 3,
    }).returning();
```

```
    // Ta bort Axfood (ska ta bort Willys och Willys Södermalm också)
    const request = new Request(`http://localhost/api/clusters/${axfood.id}`, {
      method: 'DELETE',
    });

    const response = await deleteCluster(request as any, { params: { id: axfood.id } }
);
    const data = await response.json();

    expect(data.success).toBe(true);
    expect(data.data.deletedChildrenCount).toBe(2); // Willys + Willys Södermalm

    // Verifiera att alla är borta
    const remaining = await db.query.clusterObjects.findMany({
      where: eq(clusterObjects.customerId, customer.id),
    });

    expect(remaining.length).toBe(0);
  });
});
```

E. Acceptanskriterier

- [ ] `POST /api/clusters` skapar nytt klusterobjekt
- [ ] `GET /api/clusters` listar alla (med filtrering)
- [ ] `GET /api/clusters/:id` hämtar specifikt objekt
- [ ] `PUT /api/clusters/:id` uppdaterar objekt
- [ ] `DELETE /api/clusters/:id` tar bort objekt och barn (cascade)
- [ ] `GET /api/clusters/:id/tree` hämtar hela trädet rekursivt
- [ ] Validering fungerar (felaktiga nivåer avvisas)
- [ ] 404 returneras om objekt inte hittas
- [ ] Integration tests passerar

Dokumentet fortsätter med alla återstående funktioner i Fas 1… På grund av längdbegränsningar visar jag strukturen för de återstående funktionerna.

2.4 Funktion 3: API - Metadata-nedärvning

A. Funktionsbeskrivning

Vad: Logik för att ärva metadata från förälder till barn i klusterträdet

Varför: Mats huvudprincip - “information faller nedåt”

Hur: Rekursiv funktion som traverserar trädet och ärver metadata

B. Backend Implementation

Fil: `src/lib/services/metadata-inheritance.ts`

```
import { db } from '@/db';
import { clusterObjects, metadata } from '@/db/schema';
import { eq } from 'drizzle-orm';

/**
 * Hämta all metadata för ett klusterobjekt (lokalt + ärvt)
 */
export async function getInheritedMetadata(clusterObjectId: string): Promise<Record<st
ring, any>> {
  // Hämta objektet och alla föräldrar upp till toppen
  const ancestors = await getAncestors(clusterObjectId);

  // Samla metadata från toppen och nedåt (så att barn kan överskriva förälder)
  const allMetadata: Record<string, any> = {};

  for (const ancestor of ancestors.reverse()) { // Reverse för att börja från toppen
    const metadataList = await db.query.metadata.findMany({
      where: eq(metadata.clusterObjectId, ancestor.id),
    });

    for (const meta of metadataList) {
      if (meta.inheritable) {
        allMetadata[meta.key] = JSON.parse(meta.value as string);
      }
    }
  }

  // Sist: Lägg till lokal metadata (överskriver ärvt)
  const localMetadata = await db.query.metadata.findMany({
    where: eq(metadata.clusterObjectId, clusterObjectId),
  });

  for (const meta of localMetadata) {
    allMetadata[meta.key] = JSON.parse(meta.value as string);
  }

  return allMetadata;
}

/**
 * Hämta alla föräldrar (ancestors) för ett objekt
 */
async function getAncestors(clusterObjectId: string): Promise<any[]> {
  const ancestors = [];
  let currentId: string | null = clusterObjectId;

  while (currentId) {
    const obj = await db.query.clusterObjects.findFirst({
      where: eq(clusterObjects.id, currentId),
    });

    if (!obj) break;

    ancestors.push(obj);
    currentId = obj.parentId;
  }

  return ancestors;
}

/**
 * Propagera metadata-ändring till alla barn (om inheritable)
```

```
 */
export async function propagateMetadataChange(
  clusterObjectId: string,
  metadataKey: string,
  metadataValue: any,
  inheritable: boolean
) {
  if (!inheritable) {
    // Om inte inheritable, gör ingenting
    return;
  }

  // Hämta alla barn rekursivt
  const descendants = await getDescendants(clusterObjectId);

  for (const descendant of descendants) {
    // Kontrollera om barnet har egen (överskrivande) metadata för denna key
    const localMeta = await db.query.metadata.findFirst({
      where: and(
        eq(metadata.clusterObjectId, descendant.id),
        eq(metadata.key, metadataKey),
        isNull(metadata.inheritedFrom)
      ),
    });

    if (!localMeta) {
      // Barnet har ingen lokal metadata, uppdatera ärvd
      await db.insert(metadata).values({
        clusterObjectId: descendant.id,
        key: metadataKey,
        value: JSON.stringify(metadataValue),
        category: 'logical', // Antar logical, kan förbättras
        inheritable: true,
        inheritedFrom: clusterObjectId,
      }).onConflictDoUpdate({
        target: [metadata.clusterObjectId, metadata.key],
        set: {
          value: JSON.stringify(metadataValue),
          inheritedFrom: clusterObjectId,
          updatedAt: new Date(),
        },
      });
    }
  }
}

/**
 * Hämta alla barn (descendants) för ett objekt
 */
async function getDescendants(clusterObjectId: string): Promise<any[]> {
  const descendants = [];
  const queue = [clusterObjectId];

  while (queue.length > 0) {
    const currentId = queue.shift()!;

    const children = await db.query.clusterObjects.findMany({
      where: eq(clusterObjects.parentId, currentId),
    });

    for (const child of children) {
      descendants.push(child);
      queue.push(child.id);
```

```
    }
  }

  return descendants;
}
```

C. API Endpoint

Fil: `src/app/api/clusters/[id]/metadata/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { getInheritedMetadata } from '@/lib/services/metadata-inheritance';

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  try {
    const { id } = params;
    const allMetadata = await getInheritedMetadata(id);

    return NextResponse.json({
      success: true,
      data: allMetadata,
    });
  } catch (error) {
    console.error('GET /api/clusters/:id/metadata error:', error);
    return NextResponse.json(
      {
        success: false,
        error: {
          code: 'INTERNAL_ERROR',
          message: 'Failed to fetch metadata',
        },
      },
      { status: 500 }
    );
  }
}
```

D. Testning

```
describe('Metadata Inheritance', () => {
  it('should inherit metadata from parent', async () => {
    // Skapa hierarki: Axfood → Willys → Pantrum
    const [customer] = await db.insert(customers).values({ name: 'Test' }).return-
ing();
    const [axfood] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      name: 'Axfood',
      type: 'organizational',
      level: 1,
    }).returning();

    // Lägg till metadata på Axfood
    await db.insert(metadata).values({
      clusterObjectId: axfood.id,
      key: 'prislista',
      value: '"Standard_2026"',
      category: 'logical',
      inheritable: true,
    });

    const [willys] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      parentId: axfood.id,
      name: 'Willys',
      type: 'organizational',
      level: 2,
    }).returning();

    const [pantrum] = await db.insert(clusterObjects).values({
      customerId: customer.id,
      parentId: willys.id,
      name: 'Pantrum',
      type: 'organizational',
      level: 3,
    }).returning();

    // Hämta ärvd metadata för Pantrum
    const inherited = await getInheritedMetadata(pantrum.id);

    expect(inherited.prislista).toBe('Standard_2026'); // Ärvt från Axfood
  });

  it('should allow child to override parent metadata', async () => {
    // ... Samma setup som ovan

    // Willys överskriver prislista
    await db.insert(metadata).values({
      clusterObjectId: willys.id,
      key: 'prislista',
      value: '"Willys_Special"',
      category: 'logical',
      inheritable: true,
    });

    // Pantrum ska nu ärva Willys_Special (inte Standard_2026)
    const inherited = await getInheritedMetadata(pantrum.id);

    expect(inherited.prislista).toBe('Willys_Special');
  });
});
```

2.5-2.22 Återstående Funktioner i Fas 1

På grund av längdbegränsningar listar jag strukturen för de återstående funktionerna. Varje funktion följer samma mönster som ovan: A-F (Beskrivning, Datamodell, Backend, Frontend, Testning, Accept-anskriterier)

2.5 Funktion: Orderkoncept körning (sökning + korsbefruktning)

2.6 Funktion: Uppgiftsgenerering

2.7 Funktion: Snöret - grundläggande geografisk sortering

2.8 Funktion: API - Fotografering

2.9 Funktion: Kluster-visualisering (träd)

2.10 Funktion: Kluster-editor

2.11 Funktion: Artikelregister-vy

2.12 Funktion: Orderkoncept-builder

2.13 Funktion: Uppgiftslista (admin)

2.14 Funktion: Snöret-vy (admin)

2.15 Funktion: Mobil vy för utförare

2.16 Funktion: Fotografering-gränssnitt

2.17 Funktion: Metadata-editor (mobil)

2.18 Funktion: Inloggning och dashboard

2.19 Funktion: What3words integration

2.20 Funktion: Google Maps integration

2.21 Funktion: SMS-gateway integration (grundläggande)

2.22 Funktion: Excel-import (grundläggande)

3. Tekniska Specifikationer

3.1 Komplett Databasschema med Index

(Se avsnitt 2.2 B ovan för fullständigt schema)

Viktiga Index (skapar dessa i migration):

```sql
-- Kluster-traversering
CREATE INDEX idx_cluster_parent ON cluster_objects(parent_id);
CREATE INDEX idx_cluster_customer ON cluster_objects(customer_id);
CREATE INDEX idx_cluster_type ON cluster_objects(type);
CREATE INDEX idx_cluster_level ON cluster_objects(level);

-- Metadata-sökning (KRITISKT för orderkoncept!)
CREATE INDEX idx_metadata_object ON metadata(cluster_object_id);
CREATE INDEX idx_metadata_key ON metadata(key);
CREATE INDEX idx_metadata_inheritable ON metadata(inheritable);
CREATE INDEX idx_metadata_category ON metadata(category);
CREATE INDEX idx_metadata_key_value ON metadata USING GIN (key, value); -- För JSONB-
sökning

-- Orderkoncept
CREATE INDEX idx_order_concept_customer ON order_concepts(customer_id);
CREATE INDEX idx_order_concept_cluster ON order_concepts(target_cluster_id);
CREATE INDEX idx_concept_filter_concept ON concept_filters(order_concept_id);

-- Uppgifter
CREATE INDEX idx_assignment_status ON assignments(status);
CREATE INDEX idx_assignment_date ON assignments(scheduled_date);
CREATE INDEX idx_assignment_executor ON assignments(executor_id);
CREATE INDEX idx_assignment_customer ON assignments(customer_id);
CREATE INDEX idx_assignment_cluster ON assignments(cluster_object_id);
CREATE INDEX idx_assignment_concept ON assignments(order_concept_id);

-- Foton
CREATE INDEX idx_photo_assignment ON photos(assignment_id);
CREATE INDEX idx_photo_type ON photos(type);
CREATE INDEX idx_photo_uploaded_by ON photos(uploaded_by);

-- Snöret
CREATE INDEX idx_rope_executor_week ON ropes(executor_id, week_start);
CREATE INDEX idx_rope_status ON ropes(status);
CREATE INDEX idx_rope_segment_rope ON rope_segments(rope_id);
CREATE INDEX idx_rope_segment_order ON rope_segments(rope_id, order_index);
CREATE INDEX idx_rope_segment_assignment ON rope_segments(assignment_id);
```

3.2 API-endpoints - Komplett Lista

Autentisering

```
POST   /api/auth/login              - Logga in
POST   /api/auth/logout             - Logga ut
GET    /api/auth/me                 - Hämta inloggad användare
POST   /api/auth/refresh            - Uppdatera JWT token
```

Kunder

```
GET    /api/customers               - Lista alla kunder
POST   /api/customers               - Skapa ny kund
GET    /api/customers/:id           - Hämta specifik kund
PUT    /api/customers/:id           - Uppdatera kund
DELETE /api/customers/:id           - Ta bort kund
```

Kluster

```
GET    /api/clusters                - Lista alla kluster (filtrering, pagination)
POST   /api/clusters                - Skapa nytt klusterobjekt
GET    /api/clusters/:id            - Hämta specifikt objekt (med children)
PUT    /api/clusters/:id            - Uppdatera objekt
DELETE /api/clusters/:id            - Ta bort objekt (cascade children)
GET    /api/clusters/:id/tree       - Hämta hela trädet rekursivt
GET    /api/clusters/:id/metadata   - Hämta all metadata (ärvd + lokal)
POST   /api/clusters/:id/metadata   - Lägg till metadata
PUT    /api/clusters/:id/metadata/:key - Uppdatera metadata
DELETE /api/clusters/:id/metadata/:key - Ta bort metadata
POST   /api/clusters/import         - Importera från Excel (Fas 2)
```

Artiklar

```
GET    /api/articles                - Lista alla artiklar
POST   /api/articles                - Skapa ny artikel
GET    /api/articles/:id            - Hämta specifik artikel
PUT    /api/articles/:id            - Uppdatera artikel
DELETE /api/articles/:id            - Ta bort artikel
GET    /api/articles/:id/dependencies - Hämta beroendeartiklar
POST   /api/articles/:id/dependencies - Lägg till beroende
```

Prislistor

```
GET    /api/price-lists             - Lista alla prislistor
POST   /api/price-lists             - Skapa ny prislista
GET    /api/price-lists/:id         - Hämta specifik prislista
PUT    /api/price-lists/:id         - Uppdatera prislista
DELETE /api/price-lists/:id         - Ta bort prislista
GET    /api/price-lists/:id/items   - Hämta alla artiklar i prislista
POST   /api/price-lists/:id/items   - Lägg till artikel i prislista
PUT    /api/price-lists/:id/items/:itemId - Uppdatera pris
DELETE /api/price-lists/:id/items/:itemId - Ta bort artikel från prislista
```

Orderkoncept

```
GET    /api/order-concepts          - Lista alla orderkoncept
POST   /api/order-concepts          - Skapa nytt orderkoncept
GET    /api/order-concepts/:id      - Hämta specifikt orderkoncept
PUT    /api/order-concepts/:id      - Uppdatera orderkoncept
DELETE /api/order-concepts/:id      - Ta bort orderkoncept
POST   /api/order-concepts/:id/filters - Lägg till filter
DELETE /api/order-concepts/:id/filters/:filterId - Ta bort filter
POST   /api/order-concepts/:id/preview - Förhandsgranska matchningar (UTAN att skapa
uppgifter)
POST   /api/order-concepts/:id/run  - Kör orderkoncept (SKAPA uppgifter)
```

Uppgifter

```
GET    /api/assignments             - Lista alla uppgifter (filtrering, pagination)
POST   /api/assignments             - Skapa uppgift manuellt
GET    /api/assignments/:id         - Hämta specifik uppgift
PUT    /api/assignments/:id         - Uppdatera uppgift
DELETE /api/assignments/:id         - Ta bort uppgift
PUT    /api/assignments/:id/status  - Uppdatera status
POST   /api/assignments/:id/photos  - Ladda upp foto
POST   /api/assignments/:id/extras  - Lägg till tillägg (artiklar/tid)
POST   /api/assignments/:id/start   - Starta uppgift (logga tid)
POST   /api/assignments/:id/complete - Markera uppgift som klar
GET    /api/assignments/:id/articles - Hämta alla artiklar i uppgift
```

Foton

```
GET    /api/photos                  - Lista alla foton (filtrering)
POST   /api/photos                  - Ladda upp foto
GET    /api/photos/:id              - Hämta specifikt foto
DELETE /api/photos/:id              - Ta bort foto
```

Snöret

```
GET    /api/ropes                   - Lista alla snören
POST   /api/ropes/generate          - Generera snöre för utförare/vecka
GET    /api/ropes/:id               - Hämta specifikt snöre
PUT    /api/ropes/:id               - Uppdatera snöre (omplanering)
DELETE /api/ropes/:id               - Ta bort snöre
POST   /api/ropes/:id/optimize      - Optimera rutt
GET    /api/ropes/:id/segments      - Hämta alla segment
POST   /api/ropes/:id/segments      - Lägg till segment
PUT    /api/ropes/:id/segments/:segmentId - Uppdatera segment
DELETE /api/ropes/:id/segments/:segmentId - Ta bort segment
GET    /api/executors/:id/current-week - Hämta snöre för utförare denna vecka
```

Användare

```
GET    /api/users                   - Lista alla användare
POST   /api/users                   - Skapa ny användare
GET    /api/users/:id               - Hämta specifik användare
PUT    /api/users/:id               - Uppdatera användare
DELETE /api/users/:id               - Ta bort användare
PUT    /api/users/:id/preferences   - Uppdatera preferenser
GET    /api/users/:id/assignments   - Hämta uppgifter för användare
```

3.3 Frontend-komponenter - Komplett Lista

Layout och Navigation

```
src/components/layout/
  ├── AppLayout.tsx              - Huvudlayout
  ├── Sidebar.tsx                - Sidomeny
  ├── Header.tsx                 - Toppheader
  └── MobileNav.tsx              - Mobilnavigation
```

Autentisering

```
src/components/auth/
  ├── LoginForm.tsx              - Inloggningsformulär
  └── ProtectedRoute.tsx         - Route guard
```

Kluster

```
src/components/clusters/
  ├── ClusterTree.tsx            - Träd-visualisering (react-d3-tree)
  ├── ClusterList.tsx            - Listvy
  ├── ClusterCard.tsx            - Kort för enskilt objekt
  ├── ClusterEditor.tsx          - Redigera objekt
  ├── ClusterCreator.tsx         - Skapa nytt objekt
  ├── MetadataEditor.tsx         - Redigera metadata
  └── MetadataList.tsx           - Lista metadata
```

Artiklar

```
src/components/articles/
  ├── ArticleList.tsx            - Lista artiklar
  ├── ArticleEditor.tsx          - Redigera artikel
  ├── ArticleCreator.tsx         - Skapa ny artikel
  └── ArticleDependencies.tsx    - Visa/redigera beroenden
```

Orderkoncept

```
src/components/order-concepts/
  ├── OrderConceptList.tsx       - Lista orderkoncept
  ├── OrderConceptBuilder.tsx    - Bygg orderkoncept (GUI)
  ├── FilterEditor.tsx           - Redigera filter
  ├── PreviewResults.tsx         - Förhandsgranska matchningar
  └── RunConceptButton.tsx       - Knapp för att köra koncept
```

Uppgifter

```
src/components/assignments/
  ├── AssignmentList.tsx         - Lista uppgifter (admin)
  ├── AssignmentCard.tsx         - Kort för uppgift
  ├── AssignmentDetail.tsx       - Detaljvy
  ├── AssignmentStatusBadge.tsx  - Status-badge
  ├── PhotoUploader.tsx          - Ladda upp foto
  └── ExtrasEditor.tsx           - Lägg till tillägg
```

Snöret

```
src/components/rope/
  ├── RopeView.tsx               - Övergripande vy (admin)
  ├── RopeMap.tsx                - Karta med rutt
  ├── RopeTimeline.tsx           - Tidslinje för vecka
  ├── SegmentCard.tsx            - Kort för segment
  └── OptimizeButton.tsx         - Knapp för optimering
```

Mobil (Utförare)

```
src/components/mobile/
  ├── ExecutorDashboard.tsx      - Dashboard för utförare
  ├── TodaysList.tsx             - Dagens uppgifter
  ├── AssignmentMobileCard.tsx   - Uppgiftskort (mobil)
  ├── PhotoCapture.tsx           - Fotografering
  ├── MetadataQuickEdit.tsx      - Snabbredigera metadata
  └── Navigation.tsx             - Navigation till nästa plats
```

Gemensamma

```
src/components/common/
  ├── Button.tsx                 - Knapp-komponent
  ├── Input.tsx                  - Input-komponent
  ├── Select.tsx                 - Select-komponent
  ├── Modal.tsx                  - Modal-komponent
  ├── Loading.tsx                - Loading spinner
  ├── ErrorMessage.tsx           - Felmeddelande
  └── SuccessMessage.tsx         - Framgångsmeddelande
```

---

4. Implementationsordning

4.1 Vecka 1: Databas och Grundstruktur

Mål: Få databasstrukturen på plats och grundläggande API

Steg:
1. ✅ Skapa Drizzle schema (`src/db/schema.ts`)
2. ✅ Konfigurera Drizzle Kit (`drizzle.config.ts`)
3. ✅ Generera och köra migrations
4. ✅ Verifiera i Drizzle Studio
5. ✅ Skapa test-data (seed script)
6. ✅ Implementera CRUD API för kluster (`/api/clusters`)
7. ✅ Skriva unit tests för API
8. ✅ Dokumentera API-endpoints

Leverans vecka 1:
- [ ] Databas med alla tabeller
- [ ] CRUD API för kluster fungerar
- [ ] Test-data finns
- [ ] API-dokumentation påbörjad

4.2 Vecka 2: Metadata och Nedärvning

Mål: Implementera metadata-hantering och nedärvningslogik

Steg:
1. ✅ Implementera `getInheritedMetadata()` service

2. ✅ Implementera `propagateMetadataChange()` service
3. ✅ API för metadata (`/api/clusters/:id/metadata`)
4. ✅ Skriva tester för nedärvning
5. ✅ Implementera frontend MetadataEditor
6. ✅ Testa att metadata ärvs korrekt i GUI

Leverans vecka 2:
- [ ] Metadata ärvs från förälder till barn
- [ ] Barn kan överskriva föräldern metadata
- [ ] GUI för att redigera metadata
- [ ] Tester visar att nedärvning fungerar

4.3 Vecka 3: Kluster-visualisering

Mål: Bygga träd-visualisering och kluster-editor

Steg:
1. ✅ Installera och konfigurera react-d3-tree
2. ✅ Implementera ClusterTree-komponent
3. ✅ Implementera kollaps/expandera
4. ✅ Implementera zoom och pan
5. ✅ Implementera filtrering (organisatorisk/fysisk)
6. ✅ Implementera ClusterEditor
7. ✅ Implementera ClusterCreator
8. ✅ Testa med Axfood-data (5 butiker)

Leverans vecka 3:
- [ ] Träd-visualisering fungerar
- [ ] Kan skapa, redigera, ta bort objekt via GUI
- [ ] Kan lägga till metadata via GUI
- [ ] Hela Axfood Willys-strukturen (5 butiker) fungerar

4.4 Vecka 4: Artiklar och Orderkoncept

Mål: Implementera artikelregister och orderkoncept-builder

Steg:
1. ✅ API för artiklar (`/api/articles`)
2. ✅ API för prislistor (`/api/price-lists`)
3. ✅ Implementera ArticleList och ArticleEditor
4. ✅ API för orderkoncept (`/api/order-concepts`)
5. ✅ Implementera OrderConceptBuilder (GUI)
6. ✅ Implementera FilterEditor
7. ✅ Implementera preview-funktion
8. ✅ Testa med “Tvätta pantkärl”-koncept

Leverans vecka 4:
- [ ] Artikelregister fungerar
- [ ] Kan skapa orderkoncept via GUI
- [ ] Kan lägga till filter (t.ex. typ = Pantkärl)
- [ ] Preview visar matchningar

4.5 Vecka 5: Uppgiftsgenerering och Korsbefruktning

Mål: Implementera logik för att generera uppgifter från orderkoncept

Steg:
1. ✅ Implementera `runOrderConcept()` service
2. ✅ Implementera sökning i kluster baserat på filter
3. ✅ Implementera korsbefruktning (multiplicera med antal)
4. ✅ Implementera aggregering (summera enkla uppgifter)
5. ✅ API för att köra orderkoncept (`POST /api/order-concepts/:id/run`)
6. ✅ Implementera AssignmentList
7. ✅ Testa med 5 butiker (förvänta 5 uppgifter)

Leverans vecka 5:
- [ ] Orderkoncept kan köras och skapa uppgifter
- [ ] Korsbefruktning fungerar (10 pantkärl → 500 kr, 30 min)
- [ ] Aggregering fungerar (en uppgift per butik, inte 10)
- [ ] Uppgiftslista visar genererade uppgifter

4.6 Vecka 6: Snöret och Rutt-optimering

Mål: Implementera grundläggande snöre med geografisk sortering

Steg:
1. ✅ Implementera `generateRope()` service
2. ✅ Implementera geografisk sortering (enkel distansberäkning)
3. ✅ Integrera Google Maps Distance Matrix API
4. ✅ API för snöret (`/api/ropes`)
5. ✅ Implementera RopeView (admin)
6. ✅ Implementera RopeMap med rutt-visualisering
7. ✅ Testa med 5 butiker i Stockholm

Leverans vecka 6:
- [ ] Snöre genereras för utförare/vecka
- [ ] Uppgifter sorteras geografiskt
- [ ] Karta visar rutt
- [ ] Total körsträcka och tid beräknas

4.7 Vecka 7: Mobilvy och Fotografering

Mål: Bygga mobilgränssnitt för utförare

Steg:
1. ✅ Implementera ExecutorDashboard (mobil)
2. ✅ Implementera TodaysList
3. ✅ Implementera AssignmentMobileCard
4. ✅ Implementera PhotoCapture (kamera-åtkomst)
5. ✅ API för foton (`POST /api/assignments/:id/photos`)
6. ✅ Implementera beroendelogik (foto före blockerar huvuduppgift)
7. ✅ Testa på iPhone och Android

Leverans vecka 7:
- [ ] Mobilvy fungerar på iPhone och Android
- [ ] Utförare kan se dagens uppgifter
- [ ] Utförare kan ta foton före/efter
- [ ] Foton blockerar/släpper uppgifter korrekt

4.8 Vecka 8: Testning, Bugfixar och UAT

Mål: Slutföra Fas 1, fixa buggar, genomföra UAT med Mats

Steg:
1. ✅ Komplett testning av alla funktioner
2. ✅ Prestandatestning (100 objekt, 50 uppgifter)
3. ✅ Säkerhetsaudit (OWASP Top 10)
4. ✅ Bugfixar från testning
5. ✅ Förbered UAT-scenarios med Mats
6. ✅ Genomför UAT
7. ✅ Fixa kritiska buggar från UAT
8. ✅ Dokumentera Fas 1

Leverans vecka 8:
- [ ] Alla Fas 1-funktioner fungerar
- [ ] Mats godkänner systemet för pilot
- [ ] Redo att använda för 5 butiker i produktion
- [ ] Dokumentation klar

---

5. Konkreta Exempel och Use Cases

5.1 Use Case 1: Bygg Axfood Willys-kluster

Scenario: Administratören ska bygga klusterstruktur för Axfood Willys med 5 butiker

Aktör: Administratör (Mats eller Kinab-administratör)

Steg-för-steg:

1. **Logga in som administratör**
   - Gå till `https://unicorn.replit.app/login`
   - Ange email: `admin@kinab.se`
   - Ange lösenord: `********`
   - Klicka “Logga in”

2. **Skapa toppnivå (Koncern)**
   - Klicka “Kluster” i menyn
   - Klicka “Skapa nytt klusterobjekt”
   - Fyll i formulär:
     - Namn: `Axfood`
     - Typ: `Organisatorisk`

    - Level: `1`
    - Kund: `Axfood AB`
    - Klicka “Spara”

3. **Lägg till metadata på Axfood**
   - Klicka på “Axfood” i trädet
   - Klicka “Lägg till metadata”
   - Fyll i:
    - Nyckel: `prislista`
    - Värde: `Standard_2026`
    - Kategori: `Logisk`
    - Ärvs nedåt: `Ja`
    - Klicka “Spara”
4. **Skapa kedjenivå (Willys)**
   - Högerklicka på “Axfood”
   - Välj “Skapa barn”
   - Fyll i:
    - Namn: `Willys`
    - Typ: `Organisatorisk`
    - Level: `2` (auto-filled)
    - Klicka “Spara”
    - Verifiera att metadata “prislista” ärvts automatiskt
5. **Skapa distriktsnivå (Stockholm)**
   - Högerklicka på “Willys”
   - Välj “Skapa barn”
   - Fyll i:
    - Namn: `Stockholm`
    - Typ: `Organisatorisk`
    - Level: `3`
    - Klicka “Spara”
6. **Skapa butiker (5 st)**
   - För varje butik:
    - Högerklicka på “Stockholm”
    - Välj “Skapa barn”
    - Fyll i:
    - Namn: `Willys Södermalm` (osv.)
    - Typ: `Organisatorisk`
    - Level: `4`
    - Lägg till metadata:
    - `adress`: `Götgatan 100`
    - `what3words`: `index.home.raft`
    - `tidsfönster`: `06:00-10:00`
    - `kontakt_namn`: `Anna Andersson`
    - `kontakt_telefon`: `070-1234567`

    - Klicka “Spara”

7. **Skapa områden i butik (Pantrum, Avfallsrum)**
   - För varje butik:
    - Högerklicka på butiken
    - Välj “Skapa barn”
    - Fyll i:
    - Namn: `Pantrum`
    - Typ: `Organisatorisk`
    - Level: `5`
    - Lägg till metadata:
    - `kod`: `1234`
    - `gps_lat`: `59.3156`
    - `gps_lng`: `18.0649`
    - Klicka “Spara”
    - Repetera för “Avfallsrum”
8. **Skapa fysiska objekt (Pantkärl, Matavfallskärl)**
   - För varje område:
    - Högerklicka på området (t.ex. “Pantrum”)
    - Välj “Skapa barn”
    - Fyll i:
    - Namn: `Pantkärl 160L`
    - Typ: `Fysisk`
    - Level: `6`
    - Lägg till metadata:
    - `typ`: `Pantkärl`
    - `storlek`: `160L`
    - `antal`: `10`
    - Klicka “Spara”

Förväntade Resultat:
- Träd-visualiseringen visar hela strukturen: Axfood → Willys → Stockholm → 5 butiker → Pantrum/Avfallsrum → Fysiska objekt
- Metadata ärvs korrekt (prislista på alla nivåer)
- Total: 1 koncern + 1 kedja + 1 distrikt + 5 butiker + 10 områden + ~50 fysiska objekt = **~68 objekt**
- Färgkodning: Blå för organisatoriska, Grön för fysiska

Edge Cases:
- Om användaren glömmer adress på butik → varning visas
- Om användaren försöker skapa level 4 under level 2 → felmeddelande
- Om användaren tar bort butik → alla barn tas också bort (varning visas först)

5.2 Use Case 2: Skapa Orderkoncept “Tvätta pantkärl”

Scenario: Administratören skapar orderkoncept som söker efter pantkärl och skapar uppgifter

Aktör: Administratör

Steg-för-steg:

1. **Gå till Orderkoncept**
   - Klicka “Orderkoncept” i menyn
   - Klicka “Skapa nytt”

2. **Grundinformation**
   - Namn: `Tvätta pantkärl`
   - Beskrivning: `Tvättar alla pantkärl hos Willys-butiker`
   - Kund: `Axfood AB`

3. **Välj kluster att söka i**
   - Klicka “Välj kluster”
   - Träd visas
   - Klicka på “Willys” (kedjenivå)
   - Klicka “Välj”
   - Förklaring: Sökningen kommer att omfatta Willys och alla barn (alla butiker)

4. **Lägg till filter**
   - Klicka “Lägg till filter”
   - Fyll i:
     - Metadata-nyckel: `typ`
     - Operator: `Lika med (=)`
     - Värde: `Pantkärl`
     - Klicka “Lägg till”

5. **Välj artikel**
   - Klicka “Välj artikel”
   - Sök efter “Tvätta pantkärl”
   - Välj artikel:
     - Kod: `TVATT_PANTKARL`
     - Namn: `Tvätta pantkärl`
     - Pris: `50 kr/st`
     - Kostnad: `25 kr/st`
     - Tid: `3 min/st`
     - Klicka “Välj”

6. **Konfigurera multiplikation**
   - Multiplicera med metadata: `antal`
   - Förklaring: Om ett objekt har metadata “antal = 10”, skapas 1 uppgift med quantity = 10, pris = 500 kr, tid = 30 min

7. **Förhandsgranska**
   - Klicka “Förhandsgranska”
   - System visar:

```
Matchningar: 5

1. Willys Södermalm → Pantrum → Pantkärl 160L (10 st)
   → Uppgift: Tvätta pantkärl (10 st, 500 kr, 30 min)

2. Willys Vasastan → Pantrum → Pantkärl 160L (15 st)
   → Uppgift: Tvätta pantkärl (15 st, 750 kr, 45 min)

3. Willys Bromma → Pantrum → Pantkärl 160L (8 st)
   → Uppgift: Tvätta pantkärl (8 st, 400 kr, 24 min)

4. Willys Kungsholmen → Pantrum → Pantkärl 160L (12 st)
   → Uppgift: Tvätta pantkärl (12 st, 600 kr, 36 min)

5. Willys Östermalm → Pantrum → Pantkärl 160L (10 st)
   → Uppgift: Tvätta pantkärl (10 st, 500 kr, 30 min)

Totalt: 5 uppgifter, 55 pantkärl, 2 750 kr, 165 min (2h 45min)
```

8. **Kör orderkoncept**
   - Klicka “Kör koncept”
   - Bekräfta: “Är du säker? Detta kommer skapa 5 uppgifter”
   - Klicka “Ja, kör”
   - System skapar uppgifter
   - Framgångsmeddelande: “5 uppgifter skapade”

9. **Verifiera uppgifter**
   - Gå till “Uppgifter”
   - Se att 5 nya uppgifter finns
   - Status: “Ej planerad”
   - Verifiera att alla har rätt:
     - Quantity (antal kärl)
     - Estimerad pris
     - Estimerad tid

Förväntade Resultat:
- 5 uppgifter skapas (en per butik)
- Varje uppgift har korrekt quantity baserat på metadata “antal”
- Pris och tid multipliceras korrekt
- Metadata-snapshot sparas på uppgiften (adress, GPS, kod, etc.)

Edge Cases:
- Om filter inte matchar något → Preview visar “0 matchningar”
- Om artikel saknar pris → Varning visas
- Om kluster har många matchningar (>100) → Pagination i preview

---

5.3 Use Case 3: Generera Snöre för Johan (Utförare)

Scenario: Administratören genererar snöre för utförare Johan för vecka 3 (2026)

Aktör: Administratör

Steg-för-steg:

1. **Gå till Snöret**
   - Klicka “Snöret” i menyn

2. **Välj utförare och vecka**
   - Utförare: `Johan Johansson`
   - Vecka: `Vecka 3, 2026 (13-19 januari)`
   - Klicka “Generera snöre”

3. **System arbetar**
   - Hämtar alla uppgifter med status “Finplanerad” för vecka 3
   - Hämtar GPS-koordinater från metadata
   - Beräknar distansmatris med Google Maps API
   - Sorterar uppgifter geografiskt (enkel algoritm i Fas 1)
   - Skapar resor mellan uppgifter
   - Beräknar total körsträcka och tid

4. **Resultat visas**
   - Snöre skapas med:
   ```
   Måndag 13 januari:

   ◦ Resa från Kinab-kontoret → Willys Södermalm (8 km, 15 min)
   ◦ Uppgift: Tvätta pantkärl - Willys Södermalm (30 min)
   ◦ Resa → Willys Vasastan (5 km, 12 min)
   ◦ Uppgift: Tvätta pantkärl - Willys Vasastan (45 min)
   ◦ Resa → Kinab-kontoret (6 km, 14 min)

   Tisdag 14 januari:
   - Resa → Willys Bromma (12 km, 22 min)
   - Uppgift: Tvätta pantkärl - Willys Bromma (24 min)
   - Resa → Willys Kungsholmen (4 km, 10 min)
   - Uppgift: Tvätta pantkärl - Willys Kungsholmen (36 min)
   - Resa → Kinab-kontoret (5 km, 12 min)

   Onsdag 15 januari:
   - Resa → Willys Östermalm (7 km, 16 min)
   - Uppgift: Tvätta pantkärl - Willys Östermalm (30 min)
   - Resa → Kinab-kontoret (7 km, 16 min)

   Totalt:
   - Arbetstid: 165 min (2h 45min)
   - Körtid: 117 min (1h 57min)
   - Körsträcka: 54 km
   - Total tid: 282 min (4h 42min)
   ```

5. **Visualisering på karta**
   - Karta visas med alla butiker som waypoints

- Rutter ritas mellan punkterna
- Färgkodning per dag

6. **Bekräfta och spara**
   - Klicka “Bekräfta snöre”
   - Snöre sparas med status “Confirmed”
   - Johan får tillgång till snöret i sin mobilapp

Förväntade Resultat:
- Snöre skapas för Johan för vecka 3
- Uppgifter sorteras geografiskt (minimera körsträcka)
- Total körsträcka är optimal (eller nära optimal)
- Johan kan se sitt snöre i mobilappen

Edge Cases:
- Om ingen uppgift finns för veckan → Meddelande: “Inga uppgifter att planera”
- Om GPS saknas på någon butik → Varning visas, den butiken hoppas över
- Om Google Maps API misslyckas → Fallback till fågelvägen (Haversine-formel)

5.4 Use Case 4: Johan Utför Uppgift med Fotografering

Scenario: Utförare Johan utför uppgift “Tvätta pantkärl - Willys Södermalm” och dokumenterar med foton

Aktör: Utförare (Johan)

Steg-för-steg:

1. **Logga in på mobilapp**
   - Öppna app på iPhone: `https://unicorn.replit.app/mobile`
   - Logga in:
     - Email: `johan@kinab.se`
     - Lösenord: `********`
2. **Se dagens uppgifter**
   - Dashboard visar:
   ```
   Idag (Måndag 13 januari):

   ☐ Tvätta pantkärl - Willys Södermalm
   Kl 09:00 - 09:30
   Götgatan 100
   Navigera →

   ☐ Tvätta pantkärl - Willys Vasastan
   Kl 10:00 - 10:45
   Odengatan 50
   Navigera →
   ```

3. **Klicka på första uppgiften**
   - Uppgiftskort öppnas:
   ```
   Tvätta pantkärl - Willys Södermalm

   Detaljer:
   - 10 pantkärl, 160L
   - Adress: Götgatan 100
   - Kod: 1234
   - Tidsfönster: 06:00-10:00

   Uppgifter:
   ☐ Ta foto före (MÅSTE)
   ☐ Tvätta 10 pantkärl (30 min)
   ☐ Ta foto efter (MÅSTE)
   ```

4. **Försök markera klar (utan foto)**
   - Johan klickar “Markera som klar”
   - System visar felmeddelande: “Du måste ta foto före först!”

5. **Ta foto före**
   - Johan klickar “Ta foto före”
   - Kamera öppnas
   - Johan tar foto av pantrummet (innan tvätt)
   - Foto laddas upp
   - System visar: “Foto före sparat ✓”
   - Nu är “Tvätta 10 pantkärl” upplåst

6. **Markera huvuduppgift som klar**
   - Johan klickar “Starta” på “Tvätta 10 pantkärl”
   - System loggar starttid: `09:05`
   - Johan utför arbetet (30 min)
   - Johan klickar “Klar”
   - System loggar sluttid: `09:35`
   - Faktisk tid: 30 min ✓

7. **Försök slutföra (utan foto efter)**
   - Johan klickar “Slutför uppgift”
   - System visar: “Du måste ta foto efter först!”

8. **Ta foto efter**
   - Johan klickar “Ta foto efter”
   - Kamera öppnas
   - Johan tar foto av pantrummet (efter tvätt)
   - Foto laddas upp
   - System visar: “Foto efter sparat ✓”

9. **Slutför uppgift**
   - Johan klickar “Slutför uppgift”
   - System uppdaterar:
     - Status: `Completed`

    - Faktisk starttid: `09:05`
    - Faktisk sluttid: `09:35`
    - Faktisk tid: `30 min`
    - Framgångsmeddelande: “Uppgift slutförd! ✓”

10. **Lägg till anteckning (optional)**
    - Johan klickar “Lägg till anteckning”
    - Skriver: “Två kärl behöver nytt lock”
    - Anteckning sparas som ologisk metadata

Förväntade Resultat:
- Uppgift markeras som “Completed”
- Två foton (före + efter) sparade och kopplade till uppgiften
- Faktisk tid loggad (30 min)
- Administratör kan se foton och faktisk tid i backend

Edge Cases:
- Om foto inte laddas upp (dålig täckning) → Sparas lokalt, synkas senare
- Om Johan glömmer ta foto efter → Uppgift kan inte slutföras
- Om faktisk tid blir mycket längre än estimerad (45 min vs 30 min) → Varning till admin

---

5.5 Use Case 5: Mats Ser Lönsamhet och Fakturerar

Scenario: Administratör Mats granskar slutförda uppgifter och skapar faktura

Aktör: Administratör (Mats)

Steg-för-steg:

1. **Gå till Uppgifter**
   - Klicka “Uppgifter” i menyn
   - Filtrera: Status = “Completed”
   - Sorteras efter “completed_at” (nyaste först)

2. **Se översikt**
   - Lista visar:
   ```
   5 slutförda uppgifter (Vecka 3, 2026):

     1. Tvätta pantkärl - Willys Södermalm
        Estimerat: 500 kr, 30 min
        Faktiskt: 500 kr, 30 min
        Lönsamhet: ✓ 100%

     2. Tvätta pantkärl - Willys Vasastan
        Estimerat: 750 kr, 45 min
        Faktiskt: 750 kr, 50 min
        Lönsamhet: ⚠️ 89% (5 min längre tid)

     3. Tvätta pantkärl - Willys Bromma
        Estimerat: 400 kr, 24 min
   ```

   Faktiskt: 400 kr, 24 min
   Lönsamhet: ✓ 100%

4. Tvätta pantkärl - Willys Kungsholmen
   Estimerat: 600 kr, 36 min
   Faktiskt: 600 kr, 36 min
   Lönsamhet: ✓ 100%

5. Tvätta pantkärl - Willys Östermalm
   Estimerat: 500 kr, 30 min
   Faktiskt: 500 kr, 30 min
   Lönsamhet: ✓ 100%

Totalt: 2 750 kr, 165 min (estimerat) vs 170 min (faktiskt)
Lönsamhet: 97%
```

3. **Granska foton**
   - Klicka på uppgift “Willys Vasastan” (den som tog längre tid)
   - Se foton före/efter
   - Se anteckning från Johan: “Två kärl behöver nytt lock”
   - Notera: Bra dokumentation, förklarar extra tid

4. **Markera uppgifter för fakturering**
   - Markera alla 5 uppgifter
   - Klicka “Markera för fakturering”
   - Status uppdateras: `Ready for invoicing`

5. **Skapa faktura** (manuellt i Fas 1, automatiskt i Fas 3)
   - Klicka “Skapa faktura”
   - System genererar följesedel PDF:
   ```
   FÖLJESEDEL

   Kund: Axfood AB, Willys
   Period: Vecka 3, 2026

   Utförda arbeten:
   1. Tvätta pantkärl - Willys Södermalm (10 st) .... 500 kr
   2. Tvätta pantkärl - Willys Vasastan (15 st) ..... 750 kr
   3. Tvätta pantkärl - Willys Bromma (8 st) .... 400 kr
   4. Tvätta pantkärl - Willys Kungsholmen (12 st) .. 600 kr
   5. Tvätta pantkärl - Willys Östermalm (10 st) .... 500 kr

   Totalt exkl. moms: 2 750 kr
   Moms 25%: 687,50 kr
   Totalt inkl. moms: 3 437,50 kr

   Foton bifogade: 10 st (5 före, 5 efter)
   ```

6. **Exportera till ekonomisystem** (manuellt i Fas 1)
   - Ladda ner följesedel som PDF
   ```

- Skapa faktura manuellt i Fortnox
- Markera uppgifter som “Invoiced” i Unicorn

Förväntade Resultat:
- Mats ser tydlig översikt över lönsamhet
- Foton finns som dokumentation
- Följesedel genereras automatiskt
- Faktura kan skapas (manuellt i Fas 1, automatiskt i Fas 3)

Edge Cases:
- Om faktisk tid är mycket längre än estimerad → Flagga för granskning
- Om foto saknas → Varna innan fakturering
- Om uppgift har tillägg (extra artiklar/tid) → Visa tydligt på följesedel

---

6. Viktiga Principer och Regler

6.1 Designprinciper

Enkelhet Först

- Bygg simplast möjliga version i Fas 1
- Lägg till komplexitet endast när bevisat behov finns
- “Perfect is the enemy of good”

Mats Terminologi

- Använd Mats ord: “Kluster”, “Snöre”, “Korsbefruktning”, “Gula lappar”
- INTE generiska termer: “Hierarchy”, “Route”, “Multiplication”, “Tags”

Mobile-First för Utförare

- Mobilgränssnittet är VIKTIGAST för daglig användning
- Desktop för admin är sekundärt
- Stora knappar, enkel navigation, minimal scroll

Data-Driven

- All logik baseras på metadata
- Inga hårdkodade regler
- Flexibilitet för olika kunder

Dokumentation är Affärskritiskt

- Foton före/efter är INTE optional
- Spårbarhet måste finnas på alla uppgifter
- Försäkringsärenden kräver komplett dokumentation

6.2 Affärsregler att Implementera

Kluster

Regel 1: Endast två objekttyper
- Organisatoriska (immateriella)
- Fysiska (materiella)
- INGEN tredje typ

Regel 2: Level måste vara konsekvent
- Barn måste ha parent.level + 1
- Avvisa skapande av objekt med fel level

Regel 3: Cascade delete med varning
- Ta bort förälder → tar bort alla barn
- VARNA användaren INNAN (visa antal barn som tas bort)

Metadata

Regel 4: Logisk vs Ologisk
- Logisk metadata används i systemlogik (orderkoncept, beräkningar)
- Ologisk metadata är bara information (fritext, bilder)

Regel 5: Nedärvning
- Metadata med `inheritable = true` ärvs nedåt
- Barn kan överskriva förälderns värde
- Om barn överskriver, ärv INTE vidare till barnbarn

Regel 6: Vissa metadata ärvs ALDRIG
- `antal` - Låst på objektnivå
- `typ` - Låst på objektnivå
- `serienummer` - Låst på objektnivå
- `what3words` - Exakt position (ej ärvbar)

Orderkoncept

Regel 7: Filter matchar logisk metadata
- Orderkoncept kan ENDAST matcha logisk metadata
- Ologisk metadata ignoreras i sökning

Regel 8: Korsbefruktning
- Artikel med `multiplyByMetadata = "antal"` multipliceras
- Exempel: 10 pantkärl × 50 kr = 500 kr
- Om metadata saknas → quantity = 1

Regel 9: Aggregering vs Uppdelning
- Enkla objekt (utan serienummer) → aggregera (1 uppgift med quantity = 10)
- Komplexa objekt (med serienummer) → dela upp (10 separata uppgifter)
- Regel: Om metadata “serienummer” finns → dela upp

Artiklar

Regel 10: Tre vikter för tjänster
- Weight1 = Försäljningspris (kr)

- Weight2 = Estimerad kostnad (kr)
- Weight3 = Produktionstid (minuter)

Regel 11: Tre vikter för varor
- Weight1 = Försäljningspris (kr)
- Weight2 = Inköpskostnad (kr)
- Weight3 = Ledtid (dagar)

Regel 12: Spärrar för vissa artiklar
- `lockToAddress = true` → Multipliceras INTE per objekt
- Exempel: Etablering, Avetablering, Uppblåsning
- Endast 1 gång per adress, oavsett antal objekt

Regel 13: Beroendeuppgifter måste respekteras
- `dependencyType = "before"` → Måste göras INNAN huvuduppgift
- `dependencyType = "after"` → Måste göras EFTER huvuduppgift
- `required = true` → Kan inte skippa

Uppgifter

Regel 14: Fotografering obligatorisk
- Om strukturartikel har “Foto före” → MÅSTE göras innan huvuduppgift
- Om strukturartikel har “Foto efter” → MÅSTE göras efter huvuduppgift
- Huvuduppgift kan inte markeras klar utan foto före
- Uppgift kan inte slutföras utan foto efter

Regel 15: Status-flöde
- Flöde: `not_planned` → `rough_planned` → `fine_planned` → `on_way` → `on_site` → `completed` → `ready_for_invoicing` → `invoiced`
- Kan INTE hoppa över steg (t.ex. från `not_planned` direkt till `completed`)
- Kan gå tillbaka: `fine_planned` → `rough_planned` (omplanering)

Regel 16: Tidsfönster måste respekteras
- Om kund har `customerTimeWindow` → Varna om uppgift planeras utanför
- Exempel: “Kl 06:00-10:00” → Uppgift måste startas inom detta fönster

Regel 17: Tillägg kräver motivering
- Utförare kan lägga till extra artiklar eller tid
- MÅSTE ange anledning (text)
- Administratör granskar tillägg innan fakturering

Snöret

Regel 18: Geografisk optimering
- Minimera total körsträcka
- I Fas 1: Enkel närmaste-granne-algoritm
- I Fas 3: Travelling Salesman Problem (TSP)-algoritm

Regel 19: Tidsfönster begränsar planering
- Om uppgift MÅSTE göras kl 09:00 → Kan inte planeras kl 15:00
- Snöre-optimering måste respektera tidsfönster

Regel 20: Personliga preferenser
- Utförare kan ange preferenser (hotell, arbetstider)

- Snöret föreslår baserat på preferenser
- Utförare kan justera manuellt

6.3 Kodstandarder

TypeScript

Använd Typer Överallt

```
// BRA
function createCluster(data: CreateClusterInput): Promise<ClusterObject> {
  // ...
}

// DÅLIGT
function createCluster(data: any): Promise<any> {
  // ...
}
```

Zod för Validering

```
import { z } from 'zod';

const createClusterSchema = z.object({
  name: z.string().min(1).max(255),
  type: z.enum(['organizational', 'physical']),
  level: z.number().int().min(1),
});

type CreateClusterInput = z.infer<typeof createClusterSchema>;
```

Drizzle ORM Queries

```
// BRA: Använd query API för relationer
const cluster = await db.query.clusterObjects.findFirst({
  where: eq(clusterObjects.id, id),
  with: {
    children: true,
    metadata: true,
  },
});

// OKEJ: Använd select för enklare queries
const clusters = await db.select().from(clusterOb-
jects).where(eq(clusterObjects.customerId, customerId));
```

React

Functional Components med TypeScript

```
interface ClusterCardProps {
  cluster: ClusterObject;
  onEdit?: (id: string) => void;
  onDelete?: (id: string) => void;
}

export function ClusterCard({ cluster, onEdit, onDelete }: ClusterCardProps) {
  // ...
}
```

Hooks för State Management

```
const [clusters, setClusters] = useState<ClusterObject[]>([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
```

Custom Hooks för API-anrop

```
function useClusters(customerId: string) {
  const [clusters, setClusters] = useState<ClusterObject[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetch() {
      const response = await getClusters({ customerId });
      if (response.success) {
        setClusters(response.data);
      }
      setLoading(false);
    }
    fetch();
  }, [customerId]);

  return { clusters, loading };
}
```

Naming Conventions

Files

- Components: `PascalCase.tsx` (t.ex. `ClusterTree.tsx`)
- Services: `kebab-case.ts` (t.ex. `metadata-inheritance.ts`)
- API routes: `route.ts` (Next.js konvention)

Variables

- `camelCase` för alla variabler
- `PascalCase` för typer och komponenter
- `UPPER_SNAKE_CASE` för konstanter

Functions

- Verb först: `getCluster`, `createCluster`, `updateMetadata`
- INTE: `cluster`, `metadata` (inte verb)

Error Handling

Backend

```
try {
  // ... logik
} catch (error) {
  console.error('GET /api/clusters error:', error);
  return NextResponse.json(
    {
      success: false,
      error: {
        code: 'INTERNAL_ERROR',
        message: 'Failed to fetch clusters',
        details: error instanceof Error ? error.message : undefined,
      },
    },
    { status: 500 }
  );
}
```

Frontend

```
try {
  const response = await createCluster(data);
  if (!response.success) {
    setError(response.error.message);
    return;
  }
  // ... framgång
} catch (error) {
  setError('Failed to create cluster');
  console.error(error);
}
```

6.4 Best Practices

Performance

1. Index på rätt ställen
- Foreign keys ALLTID indexerade
- Kolumner som filtreras ofta → index
- JSONB-kolumner som söks → GIN index

2. Pagination
- Lista aldrig ALLA objekt utan limit
- Default `pageSize = 50`
- Max `pageSize = 200`

3. Eager Loading
- Använd `with` i Drizzle för att hämta relationer i samma query
- Undvik N+1 queries

Security

1. Input Validering
- ALLA API endpoints måste validera input med Zod
- ALDRIG lita på input från frontend

2. Authorization
- Kontrollera att användaren har rättighet att se/ändra data
- Kunder kan bara se sina egna kluster
- Utförare kan bara se sina egna uppgifter

3. SQL Injection
- Använd ALLTID parametriserade queries (Drizzle gör detta automatiskt)
- Aldrig konkatenera strings för att bygga queries

Testing

1. Unit Tests
- Alla business logic-funktioner
- Minst 80% coverage

2. Integration Tests
- Alla API endpoints
- Använd test-databas (Testcontainers)

3. E2E Tests
- Kritiska user flows
- Minst: Skapa kluster, Skapa orderkoncept, Generera uppgift, Ta foto

---

7. Checklista för Replit 3.0

7.1 Setup och Konfiguration

- [ ] Projekt skapat i Replit
- [ ] Next.js 14+ installerat (`npx create-next-app@latest`)
- [ ] TypeScript konfigurerat
- [ ] Tailwind CSS installerat och konfigurerat
- [ ] Drizzle ORM installerat (`npm install drizzle-orm pg`)
- [ ] Drizzle Kit installerat (`npm install -D drizzle-kit`)
- [ ] PostgreSQL databas skapad (Replit tillhandahåller)
- [ ] `.env` fil skapad med `DATABASE_URL`
- [ ] Zod installerat (`npm install zod`)
- [ ] Testing setup (Jest + React Testing Library)

7.2 Databas

- [ ] Schema definierat i `src/db/schema.ts`
- [ ] Alla tabeller skapade (16 st)
- [ ] Alla enums definierade
- [ ] Alla relationer definierade
- [ ] Index skapade (minst 20 st)
- [ ] Migration genererad (`npx drizzle-kit generate:pg`)
- [ ] Migration kördes (`npx drizzle-kit push:pg`)

- [ ] Drizzle Studio verifierar tabeller (`npx drizzle-kit studio`)
- [ ] Seed script skapat för test-data
- [ ] Seed script kördes

7.3 Backend API - Kluster

- [ ] `GET /api/clusters` implementerad
- [ ] `POST /api/clusters` implementerad
- [ ] `GET /api/clusters/:id` implementerad
- [ ] `PUT /api/clusters/:id` implementerad
- [ ] `DELETE /api/clusters/:id` implementerad (cascade)
- [ ] `GET /api/clusters/:id/tree` implementerad (rekursiv)
- [ ] `GET /api/clusters/:id/metadata` implementerad
- [ ] `POST /api/clusters/:id/metadata` implementerad
- [ ] `PUT /api/clusters/:id/metadata/:key` implementerad
- [ ] `DELETE /api/clusters/:id/metadata/:key` implementerad
- [ ] Validering med Zod på alla endpoints
- [ ] Felhantering på alla endpoints
- [ ] Integration tests skrivna

7.4 Backend API - Metadata-nedärvning

- [ ] Service `getInheritedMetadata()` implementerad
- [ ] Service `propagateMetadataChange()` implementerad
- [ ] Service `getAncestors()` implementerad
- [ ] Service `getDescendants()` implementerad
- [ ] Tester för nedärvning skrivna
- [ ] Tester för överskrivning skrivna
- [ ] Edge cases testade (cirkulära referenser, etc.)

7.5 Backend API - Orderkoncept

- [ ] `GET /api/order-concepts` implementerad
- [ ] `POST /api/order-concepts` implementerad
- [ ] `GET /api/order-concepts/:id` implementerad
- [ ] `PUT /api/order-concepts/:id` implementerad
- [ ] `DELETE /api/order-concepts/:id` implementerad
- [ ] `POST /api/order-concepts/:id/filters` implementerad
- [ ] `DELETE /api/order-concepts/:id/filters/:filterId` implementerad
- [ ] `POST /api/order-concepts/:id/preview` implementerad
- [ ] `POST /api/order-concepts/:id/run` implementerad
- [ ] Service `searchClusterByFilters()` implementerad
- [ ] Service `crossPollination()` implementerad (korsbefruktning)
- [ ] Service `aggregateAssignments()` implementerad
- [ ] Tester för matching skrivna

- [ ] Tester för korsbefruktning skrivna

7.6 Backend API - Uppgifter

- [ ] `GET /api/assignments` implementerad
- [ ] `POST /api/assignments` implementerad
- [ ] `GET /api/assignments/:id` implementerad
- [ ] `PUT /api/assignments/:id` implementerad
- [ ] `DELETE /api/assignments/:id` implementerad
- [ ] `PUT /api/assignments/:id/status` implementerad
- [ ] `POST /api/assignments/:id/photos` implementerad
- [ ] `POST /api/assignments/:id/extras` implementerad
- [ ] `POST /api/assignments/:id/start` implementerad (logga tid)
- [ ] `POST /api/assignments/:id/complete` implementerad
- [ ] Beroendelogik (foto före blockerar) implementerad
- [ ] Tester för statusflöde skrivna

7.7 Backend API - Snöret

- [ ] `GET /api/ropes` implementerad
- [ ] `POST /api/ropes/generate` implementerad
- [ ] `GET /api/ropes/:id` implementerad
- [ ] `PUT /api/ropes/:id` implementerad
- [ ] `POST /api/ropes/:id/optimize` implementerad
- [ ] `GET /api/executors/:id/current-week` implementerad
- [ ] Service `generateRope()` implementerad
- [ ] Service `geographicSort()` implementerad
- [ ] Google Maps Distance Matrix integration
- [ ] Tester för generering skrivna

7.8 Frontend - Kluster

- [ ] `ClusterTree.tsx` implementerad (träd-visualisering)
- [ ] react-d3-tree konfigurerat
- [ ] Kollaps/expandera fungerar
- [ ] Zoom och pan fungerar
- [ ] Filtrering (org/fysisk) fungerar
- [ ] `ClusterList.tsx` implementerad
- [ ] `ClusterEditor.tsx` implementerad
- [ ] `ClusterCreator.tsx` implementerad
- [ ] `MetadataEditor.tsx` implementerad
- [ ] `MetadataList.tsx` implementerad
- [ ] API client (`src/lib/api/clusters.ts`) implementerad

7.9 Frontend - Artiklar

- [ ] `ArticleList.tsx` implementerad
- [ ] `ArticleEditor.tsx` implementerad
- [ ] `ArticleCreator.tsx` implementerad
- [ ] API client (`src/lib/api/articles.ts`) implementerad

7.10 Frontend - Orderkoncept

- [ ] `OrderConceptList.tsx` implementerad
- [ ] `OrderConceptBuilder.tsx` implementerad (GUI)
- [ ] `FilterEditor.tsx` implementerad
- [ ] `PreviewResults.tsx` implementerad
- [ ] `RunConceptButton.tsx` implementerad
- [ ] API client (`src/lib/api/order-concepts.ts`) implementerad

7.11 Frontend - Uppgifter

- [ ] `AssignmentList.tsx` implementerad (admin)
- [ ] `AssignmentCard.tsx` implementerad
- [ ] `AssignmentDetail.tsx` implementerad
- [ ] `AssignmentStatusBadge.tsx` implementerad
- [ ] `PhotoUploader.tsx` implementerad
- [ ] `ExtrasEditor.tsx` implementerad
- [ ] API client (`src/lib/api/assignments.ts`) implementerad

7.12 Frontend - Snöret

- [ ] `RopeView.tsx` implementerad (admin)
- [ ] `RopeMap.tsx` implementerad (Google Maps/Leaflet)
- [ ] `RopeTimeline.tsx` implementerad
- [ ] `SegmentCard.tsx` implementerad
- [ ] `OptimizeButton.tsx` implementerad
- [ ] API client (`src/lib/api/ropes.ts`) implementerad

7.13 Frontend - Mobil (Utförare)

- [ ] `ExecutorDashboard.tsx` implementerad
- [ ] `TodaysList.tsx` implementerad
- [ ] `AssignmentMobileCard.tsx` implementerad
- [ ] `PhotoCapture.tsx` implementerad (kamera-åtkomst)
- [ ] `MetadataQuickEdit.tsx` implementerad
- [ ] `Navigation.tsx` implementerad (till What3words)
- [ ] PWA-manifest konfigurerat
- [ ] Service Worker för offline-support (basic)
- [ ] Testat på iPhone Safari

- [ ] Testat på Android Chrome

7.14 Integrationer

- [ ] What3words API-nyckel konfigurerad
- [ ] What3words API client implementerad
- [ ] Google Maps API-nyckel konfigurerad
- [ ] Google Maps integration (kartor)
- [ ] Google Maps Distance Matrix integration (distanser)
- [ ] SMS-gateway (Twilio) - Optional i Fas 1

7.15 Autentisering och Användare

- [ ] `POST /api/auth/login` implementerad
- [ ] `POST /api/auth/logout` implementerad
- [ ] `GET /api/auth/me` implementerad
- [ ] JWT tokens eller sessions konfigurerat
- [ ] `LoginForm.tsx` implementerad
- [ ] `ProtectedRoute.tsx` implementerad (route guard)
- [ ] Tre roller fungerar (customer, admin, executor)

7.16 Layout och Navigation

- [ ] `AppLayout.tsx` implementerad
- [ ] `Sidebar.tsx` implementerad
- [ ] `Header.tsx` implementerad
- [ ] `MobileNav.tsx` implementerad
- [ ] Responsiv design (desktop + mobil)

7.17 Testning

- [ ] Unit tests för services (minst 80% coverage)
- [ ] Integration tests för alla API endpoints
- [ ] E2E tests för kritiska flows (minst 3 st)
- [ ] Manuell testning genomförd
- [ ] UAT-scenarios förberedda

7.18 UAT med Mats

- [ ] Scenario 1: Skapa kluster - GODKÄND
- [ ] Scenario 2: Skapa orderkoncept - GODKÄND
- [ ] Scenario 3: Generera snöre - GODKÄND
- [ ] Scenario 4: Ta foto och markera klar - GODKÄND
- [ ] Kritiska buggar fixade
- [ ] Mats godkänner för pilot

7.19 Dokumentation

- [ ] README.md uppdaterad
- [ ] API-dokumentation (minst kommentarer)
- [ ] Inline-kommentarer i komplex logik
- [ ] Deployment-instruktioner

7.20 Deployment

- [ ] Deployed till Replit (staging)
- [ ] Miljövariabler konfigurerade
- [ ] Databas backup-strategi etablerad
- [ ] Redo för pilot med 5 butiker

Slutord

Detta dokument är din kompletta guide för att bygga Unicorn Fas 1. Följ strukturen, använd kodexemplen, och fokusera på MVP (Minimum Viable Product) först. Kom ihåg:

Mats huvudprinciper:
1. Klustret är enkelt, metadatan är komplex
2. Efterlikna kundens organisation
3. Dokumentation (foton) är affärskritiskt
4. Automatisering + mänsklig input

Framgångskriterier:
- Fungerar för 5 butiker
- Mats säger “Ja, detta kan vi använda”
- Fotodokumentation fungerar smidigt
- Uppgifter genereras korrekt från orderkoncept

Nästa steg efter Fas 1:
- Fas 2: Multi-kund, Import, SMS-aviseringar
- Fas 3: Ekonomisystem-integration, Avancerad optimering
- Fas 4: Skalbarhet, Säkerhet, Lansering

Lycka till med bygget! 🚀

Detta dokument ska användas tillsammans med `Unicorn_Utvecklingsplan.md` för komplett översikt och planering.